

Université de Gabès
Faculté des Sciences de Gabès

Projet de Fin d'Études

Présenté à :

La Faculté des Sciences de Gabès
Département Informatique

En vue de l'obtention du diplôme de

**MASTER PROFESSIONNEL EN
CYBERSÉCURITÉ**

TITRE DU PROJET :

**Conception d'une couche d'agents IA intelligents avec
Retrieval-Augmented Generation (RAG) et le Model
Context Protocol (MCP) pour la plateforme éducative
SprintUp**

Réalisé par :

Wajih Zouinkhi
Hamdi Soudani

Soutenu le .../.../...

Devant le jury composé de :

M.	Président
M.	Examineur
M. Fathi Mguis	Encadrant
M. Amine Lamine	Invité

Dédicace

*À mes chers parents,
qui m'ont toujours soutenu(e) dans chacune de mes démarches...*

*À ma famille et mes amis,
pour leur présence et leurs encouragements constants...*

À tous ceux qui croient en l'éducation comme vecteur de changement.

Remerciements

Nous tenons à exprimer notre profonde gratitude à **M. Fathi Mguis**, notre encadrant académique, pour son encadrement bienveillant, ses conseils avisés et sa disponibilité tout au long de ce projet de fin d'études.

Nous remercions chaleureusement **M. Amine Lamine**, notre encadrant au sein de l'entreprise, pour son suivi régulier, ses retours constructifs et la confiance qu'il nous a accordée dans la réalisation de ce travail.

Nos remerciements s'adressent également à toute l'équipe de **SprintUp / Intellect Education** pour leur accueil chaleureux, le partage de leurs infrastructures techniques et leur soutien tout au long de cette expérience professionnelle enrichissante.

Nous remercions l'ensemble du corps enseignant du **Master Professionnel en Cybersécurité** pour la qualité de la formation reçue, en particulier les enseignements en cybersécurité et en intelligence artificielle qui ont largement nourri l'analyse présentée dans ce rapport.

Enfin, un merci sincère à nos familles et à nos camarades de promotion pour leur soutien indéfectible et leurs encouragements tout au long de cette aventure académique.

Résumé

Ce Projet de Master Professionnel présente la conception logique d'une couche d'agents [Intelligence Artificielle \(IA\)](#) destinée à la plateforme éducative SprintUp. Le travail livre une architecture indépendante de toute pile technique particulière, que la plateforme cible pourra ensuite implémenter sur sa propre infrastructure.

L'objet du document se concentre sur deux contributions de fond. La première est un **mécanisme d'indexation** des activités existantes, qui permet à un agent de vérifier qu'une activité similaire n'a pas déjà été produite avant d'en générer une nouvelle. La seconde est la conception d'un **serveur [Model Context Protocol \(MCP\)](#)** qui expose les opérations de lecture et d'écriture du syllabus sous la forme d'un catalogue d'outils typés, et la définition d'un **catalogue d'agents** consommateurs de ce serveur, chacun restreint par une liste blanche d'outils qui matérialise le principe du moindre privilège.

Le document s'organise en deux chapitres. Une introduction situe le projet dans son contexte. Le chapitre 1 présente les motifs d'orchestration d'agents, l'augmentation par recherche et le Model Context Protocol. Le chapitre 2 détaille la conception : indexation vectorielle des activités, protocole [MCP](#) appliqué au système et catalogue des agents retenus. Une conclusion générale clôt le document.

Mots-clés : IA générative, Model Context Protocol, indexation sémantique, agents conversationnels, EdTech.

Abstract

This Master’s project presents the logical design of an intelligent AI agent layer for the SprintUp educational platform. The work delivers an architecture that is independent of any particular technology stack, so that the target platform can implement it on its own infrastructure.

The document focuses on two core contributions. The first is an indexing mechanism for existing activities, which lets an agent check whether a similar activity has already been produced before generating a new one. The second is the design of a Model Context Protocol (MCP) server that exposes the read and write operations on the syllabus as a catalog of typed tools, together with a catalog of agent roles that consume this server, each restricted by a tool whitelist that materializes the principle of least privilege.

The document is organized in two chapters. The introduction positions the project in context. Chapter 1 presents the agent orchestration patterns, retrieval augmentation, and an introduction to the Model Context Protocol. Chapter 2 details the design: vector-based indexing of pedagogical activities, the [MCP](#) protocol as applied to the system, and the catalog of agents retained for this first version. A general conclusion closes the document.

Keywords: Generative AI, Model Context Protocol, semantic indexing, conversational agents, EdTech.

Table des matières

Dédicace	i
Remerciements	ii
Résumé	iii
Abstract	iv
Liste des figures	viii
Liste des tableaux	ix
Liste des abréviations	x
Introduction	1
1 Introduction à l’orchestration d’agents et au Model Context Protocol	5
1.1 Comprendre l’orchestration d’agents	5
1.1.1 Évolution des schémas d’orchestration	6
1.1.2 Comparaison des principaux cadres d’agents	6
1.1.3 LangGraph et environnement d’exécution basé sur des graphes .	7
1.1.4 Approfondissement du schéma de superviseur LangGraph	9
1.2 Le Model Context Protocol (MCP) : définition, origines et justification	10
1.2.1 Architecture détaillée et flux protocolaire	11
1.2.2 Fonctionnalités de sécurité et risques	12
1.3 État de l’art : applications en éducation	13
1.3.1 RAG agentique pour l’apprentissage personnalisé	14
1.4 Conclusion du chapitre	14
1.4.1 Synthèse et lien avec SprintUp	14
2 Conception de la couche d’agents	16
2.1 Indexation des activités pédagogiques	16
2.1.1 Le problème de la redondance	16
2.1.2 Représentation vectorielle des activités	17

2.1.3	Pipeline d'indexation	18
2.1.4	Place de l'indexation dans l'architecture	18
2.2	Le protocole MCP appliqué au système	19
2.2.1	Rappel : qu'est-ce que le MCP	19
2.2.2	Architecture MCP du système	20
2.2.3	Catalogue d'outils MCP	20
2.2.4	Principe du moindre privilège	22
2.3	Catalogue des agents	23
2.3.1	Cadre d'analyse : modes d'interaction et stratégies d'orchestration	24
2.3.2	Agent 1 : Générateur d'activités sans MCP	26
2.3.3	Agent 2 : Générateur d'activités avec MCP et indexation	27
2.3.4	Agent 3 : Création de syllabus — quatre variantes implémentées	29
2.3.5	Agent 4 : Tuteur	36
2.3.6	Agent 5 : Assistant élève	37
2.3.7	Extension envisagée : assistant administrateur	38
2.4	Lecture comparative des implémentations	38
2.4.1	Axes de comparaison	39
2.4.2	Agent 1 contre agent 2 : la valeur conditionnelle de l'ancrage	39
2.4.3	Quatre variantes de l'agent 3, quatre profils d'intégration	40
2.4.4	Coûts observés et enveloppes d'usage	41
2.4.5	Enseignements croisés	42
2.5	Conclusion : du catalogue à l'orchestration	42
3	Stratégie d'orchestration et observabilité de la couche d'agents	44
3.1	Stratégie d'orchestration des variantes	45
3.1.1	Trois surfaces d'interaction	45
3.1.2	Politique de routage	47
3.1.3	Sous-graphes communs et mutualisation	49
3.1.4	Mutualisation du contexte et moment de l'ancrage	50
3.1.5	Composabilité limitée	51
3.1.6	Synthèse : une stratégie, non une orchestration unique	52
3.2	Observabilité de la couche d'agents	53
3.2.1	Trois artefacts d'observation	53
3.2.2	Corrélation par identifiant de fil	54
3.2.3	Mesures opérationnelles utiles	55
3.2.4	Diagnostic par remontée	57
3.2.5	Ce que l'observabilité permet de défendre	58
3.3	Limites actuelles et besoins de fiabilisation	58
3.3.1	Limites du routage et de la composition	58

3.3.2	Limites de la gestion d'état et de la reprise	59
3.3.3	Limites de l'observabilité et des métriques	60
3.4	Bilan du chapitre et transition vers la sécurité applicative	60
	Conclusion générale et perspectives	63
	Références bibliographiques	66

Table des figures

1	Évolution de l'IA générative : jalons clés	2
1.1	Comparaison des principaux frameworks d'agents IA (2025–2026) . . .	6
1.2	Logo officiel de LangGraph.	7
1.3	Schéma séquentiel d'orchestration	7
1.4	Schéma parallèle d'orchestration	8
1.5	Schéma supervisé (hiérarchique) d'orchestration	9
1.6	Architecture client–serveur MCP (spécification 2025)	12
2.1	Transformation d'une activité en vecteur numérique.	17
2.2	Pipeline d'indexation : création (gauche) et recherche (droite).	18
2.3	Trois serveurs MCP et les agents qui les consomment.	20
2.4	Quatre variantes de l'agent 3 proposées pour SprintUp, positionnées au croisement du mode d'interaction utilisateur (Version 1 vs. Version 2) et de la stratégie d'orchestration (monolithique vs. sous-agents spécialistes). Chaque cellule porte le nom de code de la variante dans le dépôt et résume son schéma d'orchestration.	25
2.5	Machine d'état publique d'une activité en Version 2 (colonne <code>unit_activities.status</code>). Les états <code>ready</code> , <code>failed</code> et <code>blocked</code> sont terminaux (grisés). Le rebouclage <code>failed</code> → <code>queued</code> matérialise le retry manuel déclenché par l'enseignant ; la transition pointillée <code>failed</code> → <code>blocked</code> matérialise la propagation d'échec aux activités aval dans le graphe de dépendances <code>depends_on</code>	35
3.1	Cadre de décision du routage : trois familles de signaux (intention déclarée, contexte de session, contrat d'intégration) déterminent la variante invoquée. Le cadre est aujourd'hui appliqué de manière explicite par l'acteur ; sa formalisation prépare un routage assisté laissé en perspective.	48
3.2	Chronologie d'un fil. Une demande unique se déploie sur trois plans corrélés par un même identifiant de fil : la requête de l'acteur, la trace d'exécution (phases et appels d'outils) et la machine d'état publique des activités. La superposition permet de rattacher chaque état observable à la phase qui l'a produit.	55

Liste des tableaux

- 2.1 Outils MCP du serveur Curriculum. 21
- 2.2 Outils MCP du serveur Données étudiants. 22
- 2.3 Outils MCP du serveur Analytique. 22
- 2.4 Liste blanche d’outils MCP par agent. 23

Liste des abréviations

API Application Programming Interface.

BD Base de Données.

HITL Human-in-the-Loop.

HTTP HyperText Transfer Protocol.

IA Intelligence Artificielle.

JSON JavaScript Object Notation.

LLM Large Language Model.

MCP Model Context Protocol.

RAG Retrieval-Augmented Generation.

RBAC Role-Based Access Control.

RPC Remote Procedure Call.

SDK Software Development Kit.

SSE Server-Sent Events.

VFS Virtual File System (système de fichiers virtuel).

Introduction

À une époque où la technologie reconfigure chaque aspect de l'apprentissage et du travail, l'intelligence artificielle générative s'impose comme l'une des forces les plus transformatrices de notre temps. Ce qui a commencé par des systèmes expérimentaux capables de produire des textes ou des images simples a évolué vers des outils sophistiqués capables de créer du contenu, de raisonner sur des problèmes complexes et d'interagir de manière dynamique avec des systèmes du monde réel. Pour un étudiant en master professionnel de cybersécurité, explorer l'IA générative ne relève pas d'une simple curiosité académique. Il s'agit d'une opportunité concrète pour répondre à des enjeux réels en technologie éducative, en particulier dans un contexte d'accélération de la transformation numérique des établissements scolaires.

Le parcours de l'IA générative remonte plus loin qu'on ne le pense souvent. Ses racines se trouvent dans les premières expériences computationnelles du milieu du XX^e siècle, lorsque les chercheurs ont commencé à imaginer des machines capables d'imiter la créativité humaine. Dès les années 1960, des chatbots rudimentaires comme ELIZA ont mis en évidence le potentiel des machines à générer des réponses conversationnelles, même si elles s'appuyaient sur une simple correspondance de motifs plutôt que sur une véritable compréhension.

La véritable rupture est intervenue dans les années 2010 avec l'essor des techniques d'apprentissage profond. En 2014, Ian Goodfellow et ses collègues ont introduit les réseaux antagonistes génératifs (GANs), un cadre dans lequel deux réseaux neuronaux entrent en compétition pour produire des données de plus en plus réalistes, qu'il s'agisse d'images, d'audio ou de texte [1]. Cette innovation a ouvert la voie à la production de contenu synthétique de haute qualité et posé les principes fondateurs des modèles ultérieurs.

Le rythme s'est ensuite accéléré avec l'introduction de l'architecture Transformer en 2017. L'article « Attention Is All You Need » d'Ashish Vaswani et de son équipe a révolutionné le traitement automatique du langage en remplaçant les réseaux récurrents par des mécanismes d'auto-attention capables de traiter les séquences en parallèle et

de capturer beaucoup plus efficacement les dépendances à longue portée [2]. Cette architecture est devenue la colonne vertébrale de la série GPT développée par OpenAI. GPT-1 est apparu en 2018, suivi de GPT-2 en 2019 puis du modèle beaucoup plus volumineux GPT-3 en 2020. Chaque itération a augmenté le nombre de paramètres et l'ampleur des données d'entraînement, produisant des sorties d'apparence de plus en plus humaine.

L'explosion auprès du grand public a eu lieu en novembre 2022 avec ChatGPT, une interface conversationnelle construite sur GPT-3.5 qui a fait entrer l'IA générative dans des millions de foyers et de salles de classe du jour au lendemain. Tout à coup, n'importe qui pouvait générer des essais, du code, des plans de cours ou des histoires créatives à partir d'une simple demande en langage naturel. En 2023 et 2024, le domaine a dépassé le cadre des simples chatbots mono-modèle pour s'orienter vers des systèmes multimodaux et, surtout, vers l'IA agentique : des systèmes capables de planifier, d'utiliser des outils et d'interagir avec des environnements externes. Des modèles comme GPT-4o, Claude ou Grok ont intégré des capacités de vision, de voix et de raisonnement.

Toutefois, l'augmentation d'échelle seule s'est révélée insuffisante. Les chercheurs ont observé des limitations récurrentes : hallucinations factuelles, absence d'accès à l'information à jour et difficultés d'interaction fiable avec les systèmes logiciels existants. Ces défis ont conduit à l'émergence de techniques complémentaires, comme la Retrieval-Augmented Generation (RAG) pour ancrer les réponses dans des données réelles et, plus récemment, le Model Context Protocol (MCP). Introduit par Anthropic en novembre 2024, MCP établit une norme ouverte permettant aux modèles d'IA de découvrir et de contacter de manière sécurisée des outils, des bases de données et des services externes [3].

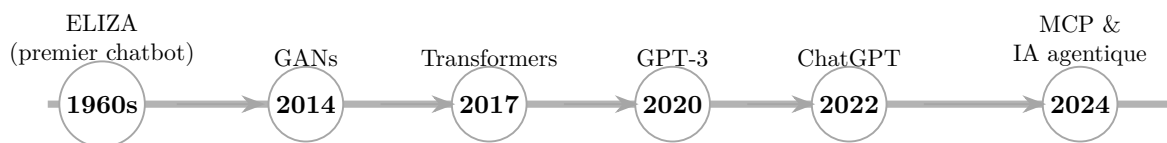


FIGURE 1 : Évolution de l'IA générative : jalons clés

L'entreprise Intellect Education a développé SprintUp, une plateforme innovante qui fonctionne comme un « Shopify de l'éducation ». Concrètement, SprintUp permet aux écoles et aux institutions de générer en quelques minutes des campus numériques entièrement fonctionnels : portails étudiants, tableaux de bord enseignants, gestion de curriculum, classes en direct et services backend exposant des API REST.

SprintUp intègre déjà une fonctionnalité de génération de syllabus fondée sur un appel direct à un modèle de langage unique (LLM). Cependant, cette approche mono-modèle présente des limitations significatives en termes de coût, de rapidité et de qualité du

contenu produit.

Problématique

L'approche actuelle de génération de curriculum sur SprintUp repose sur un **appel unique** à un **Large Language Model (LLM)** pour produire l'ensemble du contenu. Ce fonctionnement souffre de trois limitations majeures qui touchent toutes à la **qualité** du contenu produit.

Le premier problème concerne la **redondance**. Un appel mono-modèle produit du contenu nouveau à chaque exécution, sans mémoire de ce qui a déjà été produit auparavant. Sur un parcours pédagogique constitué au fil de plusieurs sessions, plusieurs activités finissent par traiter le même concept sous des angles à peine différents, ce qui dilue l'ancrage des objectifs et alourdit inutilement la charge cognitive de l'apprenant. Le deuxième porte sur l'**ancrage factuel** : en l'absence de mécanisme de contrôle, le contenu produit varie d'un appel à l'autre et aucun garde-fou n'empêche les hallucinations, les sections manquantes ou les contenus trop superficiels. Le troisième est l'**absence de séparation des privilèges** : un même prompt donne accès à toutes les opérations, ce qui rend tout usage spécialisé (révision par un tuteur, accompagnement d'un apprenant) difficile à isoler et à sécuriser.

La question qui structure ce travail est donc la suivante : *comment concevoir une famille d'agents spécialisés, intégrés de manière standardisée à une base de connaissances pédagogique partagée, qui produit un contenu sans redondance par rapport à l'existant et dont chaque agent dispose du périmètre d'accès strictement nécessaire à son rôle ?*

L'objectif central de ce projet est de remplacer l'approche mono-modèle par une **famille d'agents spécialisés**, reliés entre eux par un orchestrateur dédié et reliés à la base de données pédagogique par un serveur Model Context Protocol commun. Les agents sont au nombre de cinq dans cette première version : un **générateur d'activités sans MCP** qui sert de baseline, un **générateur d'activités avec MCP et indexation** qui consulte le contenu existant avant d'en produire, un **générateur de syllabus** qui compose ce contenu en parcours pédagogiques cohérents, un **tuteur** qui révise les productions d'apprenants et un **assistant élève** qui répond aux questions ancré dans le syllabus suivi. Un sixième agent, l'**assistant administrateur**, est mentionné en perspective.

Plus précisément, ce travail poursuit trois objectifs. Le premier est la conception d'un **mécanisme d'indexation** des activités existantes, sur lequel les agents générateurs s'appuient pour éviter de produire du contenu redondant. Le deuxième est la conception d'un **serveur MCP** qui expose ces activités et les opérations associées sous la

forme d'un catalogue d'outils typés, indépendamment de la nature des clients qui les consomment. Le troisième est la conception du **catalogue d'agents** décrivant les rôles retenus pour cette première version, avec pour chacun une liste blanche d'outils [MCP](#) qui matérialise le principe du moindre privilège. La coordination de ces agents par un orchestrateur ainsi que l'analyse de sécurité associée constituent des prolongements naturels de cette conception, qui seront traités dans des versions ultérieures de ce document.

Le document s'organise en deux chapitres. Le chapitre 1 présente les cadres d'orchestration d'agents, les techniques d'augmentation par recherche et le Model Context Protocol, qui constituent les fondations sur lesquelles s'appuie la conception du système. Le chapitre 2 détaille cette conception : indexation des activités pédagogiques, protocole [MCP](#) appliqué au système et catalogue des agents retenus pour cette première version.

En résumé, ce projet se situe à l'intersection entre l'[IA](#) générative de pointe et des besoins éducatifs concrets. En construisant une famille d'agents spécialisés autour d'une base de connaissances pédagogique partagée et d'un mécanisme d'indexation commun, nous cherchons à démontrer qu'une architecture orientée **qualité** produit un contenu plus cohérent et plus ancré que l'approche mono-modèle existante, tout en respectant les exigences de cybersécurité d'un environnement éducatif.

Chapitre 1

Introduction à l'orchestration d'agents et au Model Context Protocol

Dans la continuité de la vue d'ensemble présentée en introduction, ce chapitre établit les fondements théoriques et techniques nécessaires à la couche d'IA proposée. Nous commençons par clarifier ce que signifie l'orchestration d'agents dans les systèmes modernes d'IA générative, explorons les principaux schémas d'architecture actuellement utilisés, puis nous concentrons sur le Model Context Protocol, l'innovation clé qui rend praticable et passable à l'échelle l'intégration fiable avec des services backend existants. La discussion s'approfondit ensuite sur des mécanismes d'orchestration concrets, en particulier le schéma de superviseur LangGraph, et sur une vue détaillée de la conception protocolaire et des propriétés de sécurité de MCP. Enfin, nous introduisons le paradigme émergent de RAG agentique et passons en revue l'état de l'art des applications en éducation afin de montrer comment ces concepts alimentent directement notre implémentation SprintUp.

1.1 Comprendre l'orchestration d'agents

Fondamentalement, un agent IA est plus qu'un simple chatbot. C'est un système autonome qui perçoit son environnement (via les requêtes utilisateur ou les données récupérées), raisonne sur des objectifs, sélectionne les outils ou actions appropriés et les exécute en séquence ou en parallèle jusqu'à l'atteinte de l'objectif. Lorsque plusieurs agents ou outils doivent collaborer, ce qui est inévitable pour traiter des demandes éducatives complexes, l'orchestration devient essentielle. L'orchestration désigne la logique de contrôle qui coordonne le flux d'information, délègue les sous-tâches, gère

l'état et garantit la cohérence des résultats finaux. Sans orchestration appropriée, les agents risquent de boucler indéfiniment, d'entreprendre des actions contradictoires ou d'utiliser les ressources de façon inefficace.

1.1.1 Évolution des schémas d'orchestration

L'orchestration d'agents a évolué rapidement depuis les premiers systèmes d'enchaînement de prompts (*prompt chaining*) en 2022–2023. Les systèmes initiaux reposaient sur des pipelines linéaires. À partir de 2024, des schémas parallèles et hiérarchiques sont apparus, poussés par la nécessité de fiabilité en production. En 2025–2026, les environnements d'exécution basés sur des graphes sont devenus le paradigme dominant, permettant des branchements conditionnels, un état persistant, de l'intervention humaine dans la boucle et une replanification dynamique. Cette évolution répond directement aux limites observées dans les premiers systèmes agentiques, notamment dans les contextes éducatifs où les tâches impliquent de multiples services backend et des données étudiantes en temps réel [4], [5].

1.1.2 Comparaison des principaux cadres d'agents

Plusieurs frameworks dominent désormais l'écosystème. Le tableau ci-dessous résume les différences clés pertinentes pour notre implémentation SprintUp :

Comparison of Leading AI Agent Frameworks (2025–2026)				
Framework	Orchestration Style	Multi-Agent Support	State Management	Best For EdTech
LangGraph	Graph-based (nodes/edges)	Excellent	Persistent + checkpoints	Complex workflows
CrewAI	Role-based crews	Very strong	Memory sharing	Rapid multi-agent
AutoGen	Conversational	Strong	Asynchronous	Collaborative tutoring
Semantic Kernel	Planner + skills	Moderate	Enterprise integration	Microsoft ecosystems
LlamaIndex	RAG-focused	Moderate	Vector store	Knowledge retrieval

FIGURE 1.1 : Comparaison des principaux frameworks d'agents IA (2025–2026)

1.1.3 LangGraph et environnement d'exécution basé sur des graphes



FIGURE 1.2 : Logo officiel de LangGraph.

LangGraph (au sein de l'écosystème LangChain) illustre l'orchestration moderne. Il représente les workflows sous forme de graphes orientés où les nœuds sont des agents ou des outils et les arêtes définissent le routage conditionnel. Un état persistant et des points de contrôle avec intervention humaine garantissent la fiabilité, un aspect critique lors de l'intégration de données étudiantes sensibles [5]. Dans notre couche SprintUp, nous tirerons parti du schéma de superviseur de LangGraph combiné à l'exposition d'outils via MCP.

Trois principaux schémas d'orchestration dominent la pratique actuelle et sont bien pris en charge par des cadres comme LangGraph. La figure 1.3 présente le premier schéma.

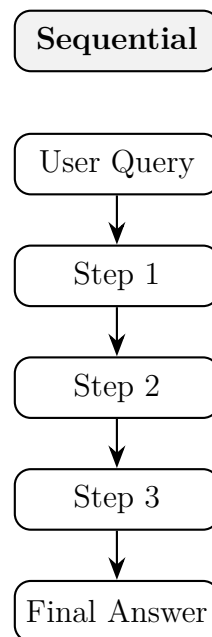


FIGURE 1.3 : Schéma séquentiel d'orchestration

Sequential orchestration organise les tâches dans une chaîne linéaire. Chaque étape attend la fin de la précédente avant de transmettre sa sortie. Ce schéma convient bien à des flux de travail clairement définis, comme « récupérer l'emploi du cours → extraire les notes du chapitre concerné → générer des exercices supplémentaires ». Sa simplicité facilite le débogage, mais il peut devenir un goulot d'étranglement lorsque des sous-tâches pourraient s'exécuter indépendamment.

La figure 1.4 présente le deuxième schéma.

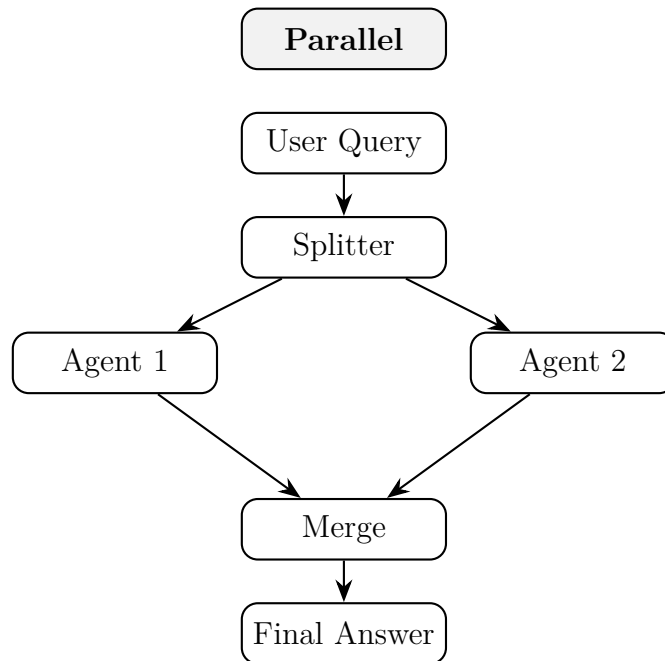


FIGURE 1.4 : Schéma parallèle d'orchestration

Parallel (ou simultaneous) orchestration permet à plusieurs agents ou outils de s'exécuter en même temps. Une requête peut lancer séparément la récupération de l'emploi du temps du jour, la recherche de ressources multimédias et la rédaction d'un résumé ; les résultats sont ensuite fusionnés par une étape de combinaison. Cette approche améliore la rapidité et convient particulièrement aux contextes éducatifs où les enseignants ont besoin d'un support rapide et multiple. La difficulté principale réside dans la résolution des conflits et la synchronisation de l'état.

La figure 1.5 présente le troisième schéma.

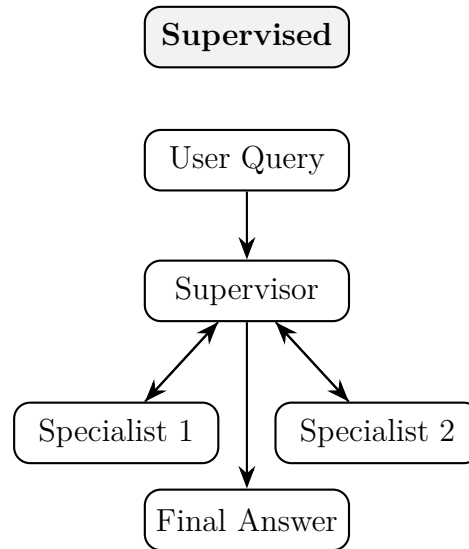


FIGURE 1.5 : Schéma supervisé (hiérarchique) d'orchestration

Supervised (ou hierarchical/orchestrator-worker) orchestration introduit un agent superviseur central qui agit comme routeur et contrôleur de qualité. Le superviseur analyse la requête utilisateur, décide quels agents spécialisés appeler, surveille l'avancement, puis synthétise la réponse finale. Ce schéma, bien implémenté dans les graphes superviseur de LangGraph, reflète la gestion d'une équipe humaine et offre de bons mécanismes de gestion d'erreur et de repli. C'est le schéma que nous privilégierons dans notre couche SprintUp, car il équilibre flexibilité et contrôle, ce qui est essentiel pour une intégration avec des services backend en production.

LangGraph montre comment ces schémas peuvent coexister dans un même runtime basé sur des graphes. Les nœuds représentent des agents ou des outils, les arêtes définissent le flux de données et le routage conditionnel, et l'état persistant avec points de contrôle humains assure la fiabilité. Ces cadres ont fait évoluer le développement d'agents, passant de chaînes de prompts fragiles à des workflows testables et de niveau production.

1.1.4 Approfondissement du schéma de superviseur LangGraph

Les environnements d'exécution basés sur des graphes comme LangGraph fournissent non seulement des schémas abstraits, mais aussi des mécanismes concrets pour construire des systèmes multi-agents robustes. Dans une configuration typique de superviseur, un nœud contrôleur de haut niveau reçoit la requête utilisateur, inspecte l'état courant et décide quels agents spécialistes ou outils invoquer ensuite [5], [6]. Ces spécialistes peuvent inclure par exemple un agent « cours », un agent « notes » ou un

générateur d'exercices, chacun responsable d'une capacité restreinte alignée sur un service backend spécifique.

Le modèle de graphe d'état de LangGraph permet au superviseur de maintenir une mémoire structurée des résultats intermédiaires, des sorties d'outils et des signaux d'erreur, ce qui facilite la replanification lorsqu'un outil échoue ou que des informations supplémentaires sont nécessaires [7]. Les mécanismes de checkpointing et les contrôles avec humain dans la boucle, désormais standards dans les versions modernes de LangGraph, jouent un rôle clé dans les scénarios éducatifs où des interventions d'enseignants ou d'administrateurs peuvent être requises avant l'exécution d'actions sensibles sur les données étudiantes [4]. Cette combinaison de routage supervisé, d'état persistant et de points de contrôle explicites constituera l'ossature de la couche d'orchestration SprintUp.

1.2 Le Model Context Protocol (MCP) : définition, origines et justification

Tandis que l'orchestration gouverne le flux de haut niveau, le Model Context Protocol traite la question plus bas niveau de la communication des agents avec les systèmes externes. Annoncé par Anthropic en novembre 2024 puis étendu via des mises à jour de spécification ouvertes, MCP est un protocole ouvert et standardisé qui fonctionne comme un « port USB-C de l'IA » [3], [8].

Il définit une architecture client–serveur cohérente. Côté client, un composant MCP intégré dans l'environnement d'exécution de l'agent charge les descriptions d'outils et leurs capacités dans la fenêtre de contexte du modèle. Le modèle décide alors quels outils appeler et émet des requêtes structurées. Côté serveur, des serveurs MCP exposent les services backend (bases de données, API, systèmes de fichiers, fonctions métier) de façon uniforme via JSON-RPC sur des canaux sécurisés. La spécification 2025 définit deux transports principaux : **stdio** pour les intégrations locales et **Streamable HTTP** (successeur du transport SSE initial) pour les déploiements réseau ; ce dernier permet des requêtes HTTP POST stateless avec réponses JSON directes, facilitant le passage à l'échelle derrière un load balancer. Les résultats transitent par le même protocole, ce qui autorise des interactions multi-tours fluides.

À la différence des mécanismes traditionnels d'appel de fonctions (*function calling*) qui exigent du code de liaison spécifique pour chaque nouveau service, MCP découple l'agent des détails d'implémentation. Les développeurs implémentent le serveur MCP une seule fois pour leur backend (dans notre cas, les endpoints de services étudiants pour les cours, notes, génération de contenu, etc.), et tout agent conforme peut ensuite

découvrir et utiliser immédiatement ces outils. MCP a précisément été conçu pour résoudre la fragmentation qui pénalisait les premiers développements d'agents : avant 2024, chaque entreprise devait écrire des intégrations sur mesure pour chaque base de données, API ou outil, un cauchemar de maintenance qui passait mal à l'échelle et introduisait des incohérences de sécurité [3], [9]. D'un point de vue cybersécurité, au cœur de ce programme, les hooks d'authentification intégrés au protocole, les périmètres d'autorisation et le format de messages propice à l'audit représentent une avancée majeure par rapport aux solutions *ad hoc* d'appel de fonctions [10].

MCP ne remplace pas la récupération d'information ; il la complète. La recherche agentique sur sources externes excelle dans la collecte de connaissances actualisées pour ancrer le contenu généré dans des sources fiables. MCP permet à l'agent d'agir sur ces connaissances en appelant des services applicatifs en temps réel. Dans le contexte SprintUp, la rédaction d'une activité déclenche d'abord la collecte du contexte pédagogique pertinent par recherche externe, puis les outils MCP de création et de mise à jour pour persister le contenu dans la base de connaissances.

1.2.1 Architecture détaillée et flux protocolaire

MCP définit une architecture client–serveur basée sur JSON-RPC via des événements envoyés par le serveur. Le client MCP (intégré dans l'exécution de l'agent) charge les descriptions d'outils dans la fenêtre de contexte du modèle et les expose sous forme de définitions structurées que le modèle peut appeler [8], [9]. Quand le modèle décide d'invoquer un outil, il émet une requête JSON-RPC :

Exemple de requête JSON-RPC MCP

```
{"jsonrpc": "2.0", "method": "tools/call",  
  "params": {"name": "create_activity",  
    "arguments": {"unit_id": "u-01",  
      "title": "Les bases de données relationnelles"}}
```

Le serveur MCP reçoit la requête, exécute la logique backend correspondante et renvoie une réponse structurée :

Exemple de réponse JSON-RPC MCP

```
{"jsonrpc": "2.0", "id": 1,  
  "result": {"id": "a-42",  
    "title": "Les bases de données relationnelles",  
    "position": 0, "status": "created"}}
```

Le transport *Streamable HTTP* se prête naturellement à un mode sans état dans

lequel chaque appel [Remote Procedure Call \(RPC\)](#) est auto-contenu et n'oblige pas le serveur à mémoriser une session de client [8]. La figure 1.6 illustre cette architecture de haut niveau.

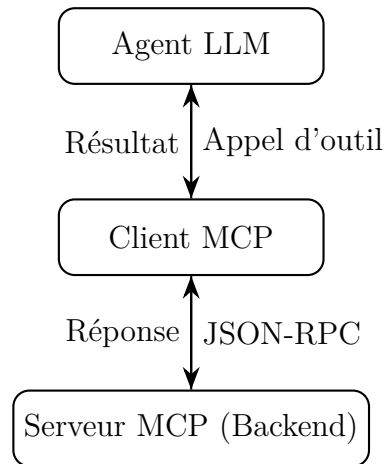


FIGURE 1.6 : Architecture client-serveur MCP (spécification 2025)

Du point de vue de l'implémentation, la spécification MCP et les implémentations de référence dans des langages courants (Python, TypeScript) fournissent une cartographie claire entre les capacités côté serveur et les schémas d'outils visibles dans le contexte du modèle [8]. Cela réduit la complexité accidentelle et facilite la compréhension des opérations qu'un agent est autorisé à effectuer dans un déploiement éducatif donné.

1.2.2 Fonctionnalités de sécurité et risques

Sur le plan de la cybersécurité, MCP présente plusieurs avantages par rapport aux schémas informels d'appel de fonctions. Tout d'abord, il encourage une définition explicite des outils disponibles et de leurs paramètres, ce qui s'aligne sur le principe du moindre privilège : un agent interagissant avec SprintUp peut n'avoir accès qu'à des méthodes de consultation de cours en lecture seule pour les profils étudiants, tandis que les agents enseignants bénéficient de capacités d'écriture plus larges [10]. Ensuite, les messages JSON-RPC structurés constituent une piste d'audit naturelle pouvant être journalisée et surveillée pour détecter des comportements suspects.

Des risques subsistent néanmoins. Des attaquants pourraient tenter d'enregistrer des serveurs MCP malveillants exposant des outils conçus pour exfiltrer des données ou injecter du contenu adversarial dans le contexte de l'agent. Des attaques par injection de prompt ou empoisonnement de données peuvent également survenir via les réponses d'outils si celles-ci ne sont pas correctement filtrées [10]. Plusieurs mesures de mitigation sont recommandées. La première consiste à exiger une authentification et une autorisation strictes des serveurs MCP avant leur mise à disposition de l'agent.

La deuxième repose sur des périmètres fins et un contrôle d'accès basé sur les rôles (RBAC) définissant les outils qu'un profil d'utilisateur peut invoquer. La troisième impose la validation des entrées et des sorties de tous les appels d'outils, avec filtrage du contenu non fiable avant qu'il n'atteigne le modèle. La quatrième, enfin, repose sur une journalisation exhaustive et une détection d'anomalies sur le trafic MCP, en particulier pour les opérations impliquant des données personnelles d'étudiants.

Dans l'implémentation SprintUp, ces mesures seront intégrées aux contrôles de sécurité existants et aux politiques de protection des données.

1.3 État de l'art : applications en éducation

La combinaison de l'orchestration, de la Retrieval-Augmented Generation (RAG) et d'intégrations de type MCP a déjà donné lieu à des résultats convaincants dans des environnements éducatifs à travers le monde. L'Arizona State University (ASU), par exemple, a lancé en 2024 un important « AI Innovation Challenge » ayant attiré plus de 175 propositions et financé plus de 200 projets couvrant la majorité de ses unités académiques [11]. Ces initiatives incluent des agents de tutorat personnalisé et des outils de simulation combinant RAG pour la récupération de connaissances et des mécanismes d'orchestration pour fournir des expériences d'apprentissage adaptatives.

De la même façon, des prototypes basés sur RAG ont montré de bonnes performances dans les environnements universitaires. Une revue systématique récente sur RAG pour les applications éducatives met en évidence la façon dont les modèles augmentés par la récupération peuvent alimenter des systèmes d'apprentissage interactifs, la génération de contenu et l'évaluation automatisée, tout en réduisant les hallucinations par rapport aux modèles purement génératifs [12], [13].

Un exemple particulièrement pertinent pour ce projet est l'expérience d'Intellect Academy en Tunisie, qui opère via la plateforme SprintUp sur plusieurs campus et filières de formation [14]. En intégrant l'IA pour le développement de curricula, les parcours d'apprentissage adaptatifs et le pilotage unifié de la performance, l'académie a accéléré la création de contenu tout en préservant une forte maîtrise pédagogique.

L'Université Northeastern a, elle, noué un partenariat avec Anthropic pour déployer des agents basés sur Claude sur ses campus pour le tutorat personnalisé et le support administratif [15], [16]. Ces initiatives illustrent une tendance plus large en 2025–2026 : l'émergence d'universités « IA natives » où des systèmes agentiques soutiennent le conseil, le tutorat et les workflows d'alerte précoce à grande échelle [17], [18].

1.3.1 RAG agentique pour l'apprentissage personnalisé

Les pipelines classiques de Retrieval-Augmented Generation (RAG) récupèrent un ensemble statique de documents à partir de la requête utilisateur, puis génèrent une réponse en un seul passage. Si cela améliore l'ancrage factuel, ces workflows peuvent rencontrer des difficultés pour le raisonnement multi-étapes, les questions ambiguës ou les tâches nécessitant une exploration adaptative de la base de connaissances [12], [19]. Le RAG agentique étend ce paradigme en intégrant des agents autonomes dans la chaîne RAG. Ces agents s'appuient sur des schémas de conception tels que la planification, la réflexion, l'utilisation d'outils et la collaboration multi-agents pour piloter dynamiquement les stratégies de récupération, affiner itérativement le contexte et adapter les workflows [17].

Dans des scénarios d'apprentissage personnalisé, un système de RAG agentique peut ainsi planifier une séquence de récupérations à partir de notes de cours, de chapitres de manuels et d'anciens sujets d'examen, puis réfléchir à la suffisance du contexte récupéré et émettre des requêtes complémentaires en cas de lacunes. Il peut également utiliser des outils pour parcourir des graphes de connaissances ou interroger un modèle de l'étudiant afin d'adapter les explications à son niveau actuel.

La revue consacrée à RAG pour les applications éducatives met spécifiquement en avant quatre grands schémas d'application : systèmes de questions-réponses éducatifs, chatbots pédagogiques, systèmes de tutorat pilotés par IA et parcours d'apprentissage adaptatifs [12], [13]. Chacun de ces schémas s'aligne étroitement avec les services backend existants de SprintUp.

1.4 Conclusion du chapitre

Ce chapitre a posé les bases conceptuelles : l'orchestration fournit l'intelligence de coordination des tâches complexes, tandis que MCP apporte la plomberie standardisée et sécurisée qui connecte les agents aux systèmes backend réels. Combinées à RAG (détaillé dans les chapitres suivants), ces technologies constituent la pile complète nécessaire pour concrétiser notre vision pour SprintUp. Les schémas et les protocoles présentés ici ne sont pas théoriques ; ils adressent directement le déficit d'intégration mis en évidence en introduction et orienteront chacune des décisions de conception à venir.

1.4.1 Synthèse et lien avec SprintUp

Le schéma d'orchestration supervisée, combiné à l'exposition d'outils via MCP et à l'ancrage par RAG, s'ajuste naturellement aux services backend de SprintUp. L'analyse

de la sécurité de MCP et de l'orchestration d'agents présentée dans ce chapitre fournit un socle pour concevoir une couche d'agents conformes et auditables, respectant la vie privée des étudiants et les contraintes institutionnelles. Forts de ces fondations, nous pouvons désormais passer à l'implémentation concrète de la récupération d'information, de l'exposition des outils via MCP et de la logique d'orchestration dans les chapitres de mise en œuvre qui suivent.

Chapitre 2

Conception de la couche d'agents

L'introduction a posé le problème (redondance, absence d'ancrage factuel, absence de séparation des privilèges) et le chapitre 1 a établi les fondations théoriques : orchestration d'agents, Model Context Protocol et [Retrieval-Augmented Generation \(RAG\)](#) agentique. Le présent chapitre traduit ces fondations en une conception logique complète, puis relit les implémentations livrées comme un *catalogue comparé de formes d'agents*. Il prépare ainsi la stratégie d'orchestration qui fera l'objet du chapitre 3.

La première partie traite de l'indexation des activités, pierre angulaire sans laquelle la non-redondance ne serait qu'un vœu. La deuxième présente le protocole [MCP](#) tel qu'il s'applique au système, avec le catalogue d'outils qu'il expose et le principe du moindre privilège qui gouverne leur distribution. La troisième détaille le catalogue des agents retenus, en réservant une matrice à quatre variantes pour l'agent de création de syllabus. La quatrième propose une lecture comparative des implémentations : non un test de conformité, mais un examen architectural de ce que chaque variante privilégie et de ce qu'elle paie en contrepartie. La conclusion enchaîne sur le chapitre 3, en montrant que ces différences de conception appellent une stratégie d'orchestration de niveau supérieur.

2.1 Indexation des activités pédagogiques

2.1.1 Le problème de la redondance

L'approche mono-modèle décrite en introduction souffre d'un défaut structurel : chaque appel au [LLM](#) produit du contenu nouveau sans mémoire de ce qui a déjà été produit. Sur un parcours pédagogique enrichi au fil de plusieurs sessions, plusieurs activités finissent par traiter le même concept sous des angles à peine différents.

Cette redondance n'est pas qu'un problème esthétique. Elle dilue l'ancrage des objectifs d'apprentissage, alourdit la charge cognitive de l'apprenant et complique le travail de l'enseignant qui doit trier manuellement les doublons. Pour qu'un agent génère du contenu utile, il doit d'abord savoir ce qui existe.

La solution retenue est un mécanisme d'indexation qui transforme chaque activité existante en une représentation numérique comparable, et qui expose une opération de recherche par similarité aux agents générateurs. L'indexation constitue la pierre angulaire du système : sans elle, les parties suivantes (MCP, agents) perdraient leur raison d'être.

2.1.2 Représentation vectorielle des activités

Le principe est le suivant. Chaque activité pédagogique (cours, question, exercice ou quiz) est décrite par un texte : son titre concaténé avec son contenu. Ce texte est transformé en un vecteur numérique de dimension fixe par un modèle de plongement (*embedding model*). La dimension du vecteur est déterminée par le modèle retenu ; elle est sans incidence sur la logique de l'agent, qui ne manipule jamais les composantes individuelles. Deux textes sémantiquement proches, par exemple deux exercices portant sur la résolution d'équations du second degré, produisent des vecteurs proches dans cet espace, même si leur formulation diffère.

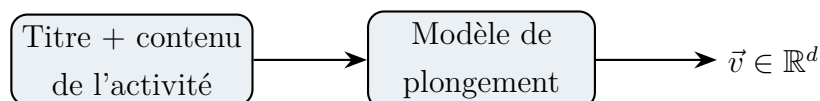


FIGURE 2.1 : Transformation d'une activité en vecteur numérique.

À partir de cette représentation, le système définit une **mesure de similarité** entre deux activités. Soit $\vec{u}$ le vecteur d'une activité existante et $\vec{v}$ celui d'un brouillon à créer ; la similarité $s(\vec{u}, \vec{v})$ est une fonction qui croît lorsque les deux vecteurs pointent dans la même direction de l'espace de plongement, et qui décroît lorsqu'ils en divergent. Le choix précis de cette mesure (cosinus, distance euclidienne ou autre) relève de la configuration du moteur d'indexation et non de la conception de l'agent.

Une calibration de cette similarité — un point au-delà duquel deux activités sont considérées comme un quasi-doublon — peut être fixée côté serveur de manière empirique. Cette calibration reste un paramètre d'implémentation : elle n'appartient pas au contrat de l'agent, qui consomme la primitive de recherche sans en connaître les constantes. Le chapitre ne fait donc pas reposer son argumentaire sur une valeur numérique particulière ; ce sont les *propriétés* de la recherche par similarité (ordonnancement par proximité, signal complémentaire de la lecture directe) qui structurent la conception.

2.1.3 Pipeline d'indexation

Le pipeline d'indexation comporte deux flux complémentaires : la **création** d'un index à partir des activités existantes, et la **recherche** par similarité avant chaque nouvelle écriture.

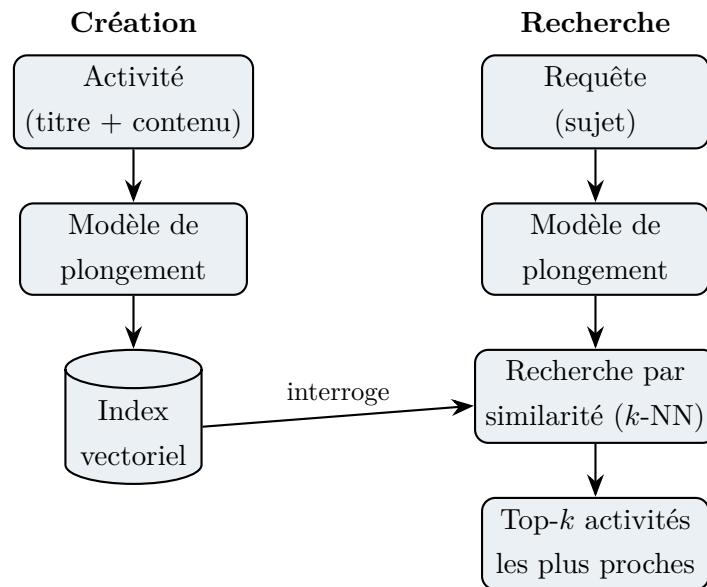


FIGURE 2.2 : Pipeline d'indexation : création (gauche) et recherche (droite).

À la création, chaque nouvelle activité passe par le modèle de plongement et le vecteur résultant est persisté dans un index vectoriel aux côtés de l'identifiant de l'activité. À la recherche, la requête de l'agent (typiquement le sujet d'une activité à créer) est elle-même plongée, puis comparée à toutes les entrées de l'index ; les k plus proches sont retournées avec leur score de similarité.

2.1.4 Place de l'indexation dans l'architecture

L'indexation n'est pas un service autonome que les agents appellent directement. Elle est encapsulée dans le serveur **MCP** sous la forme d'outils de recherche sémantique typés, accompagnés d'une mise à jour automatique de l'index à chaque création d'activité ou d'unité. L'agent ne voit donc l'indexation qu'à travers deux interfaces : une opération de recherche par similarité, en lecture, et une persistance enrichie de l'indexation, en écriture.

Deux niveaux d'usage de cette indexation se distinguent dans la conception du système.

Le premier niveau est l'usage **intra-syllabus**. Lorsqu'un agent doit produire une nouvelle activité dans un syllabus existant, son premier réflexe est la **lecture directe** des unités et activités sœurs : il interroge le serveur **MCP** pour lister la structure du

syllabus, puis lit le corps des activités voisines pour comprendre ce qui a déjà été enseigné. Cette lecture de proximité, déterministe et peu coûteuse, porte l’essentiel de la non-redondance dans le périmètre d’une seule unité. La recherche par similarité y joue le rôle de **signal complémentaire** : elle permet de détecter un quasi-doublon sémantique que la simple comparaison de titres n’aurait pas révélé, ou de signaler qu’une activité jugée nouvelle reformule en réalité un contenu existant.

Le second niveau est l’usage **global**, où la lecture directe ne peut plus suffire. Trois situations le réclament. D’abord, la création d’un **nouveau syllabus** : pour éviter qu’un enseignant ne produise un doublon d’un syllabus existant dans la base, l’agent doit pouvoir interroger l’ensemble du catalogue par similarité, et pas seulement la liste textuelle des titres. Ensuite, l’**agent assistant élève** : lorsqu’un apprenant est bloqué sur un concept, l’agent doit pouvoir retrouver les activités sémantiquement proches à recommander, indépendamment de l’unité courante. Enfin, l’**agent tuteur** : pour proposer des variantes pédagogiques ou du matériel d’appui, l’agent doit pouvoir retrouver dans l’ensemble du catalogue les activités voisines d’une activité donnée.

C’est sur ces trois usages que repose la valeur structurante de l’indexation. Elle reste accessible au niveau intra-syllabus comme garde-fou anti-doublon, mais son rôle décisif se déploie à l’échelle globale, dès que les agents doivent raisonner au-delà du périmètre d’un seul syllabus.

2.2 Le protocole MCP appliqué au système

2.2.1 Rappel : qu’est-ce que le MCP

Le Model Context Protocol (**MCP**), introduit en détail au chapitre 1, est un protocole ouvert qui standardise la façon dont un agent **LLM** découvre et appelle des outils fournis par un serveur externe [3], [9]. Son architecture client-serveur repose sur **JavaScript Object Notation (JSON)-RPC** : le client (intégré dans le runtime de l’agent) charge les descriptions d’outils, le modèle décide lesquels appeler, et le serveur exécute la logique correspondante puis renvoie un résultat structuré.

Deux propriétés du protocole sont particulièrement pertinentes pour le présent système. La première est la *découverte automatique* : un agent conforme peut interroger un serveur **MCP** pour obtenir la liste de ses outils, y compris leurs schémas d’entrée et leur documentation, ce qui permet d’ajouter un nouvel outil au serveur sans modifier le code de l’agent — un simple redémarrage de la connexion suffit. La seconde est le *découplage entre l’agent et son backend* : l’agent ne connaît ni la base de données ni le langage du serveur, mais seulement des noms d’outils et des schémas **JSON**, ce

qui permet de faire évoluer le backend (changer de [Base de Données \(BD\)](#), ajouter un cache) sans impacter les agents qui le consomment.

2.2.2 Architecture MCP du système

Le système expose trois serveurs [MCP](#), chacun couvrant un domaine fonctionnel distinct (figure 2.3).

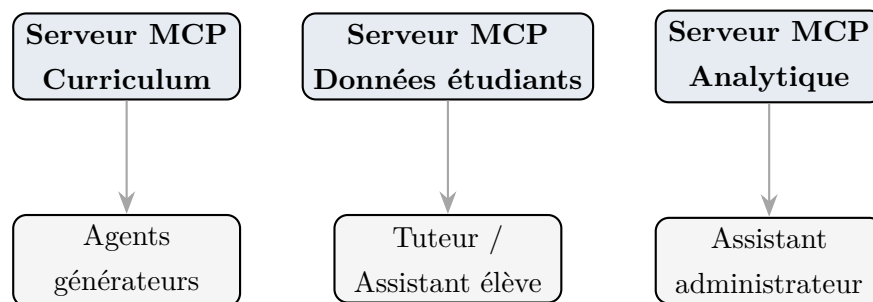


FIGURE 2.3 : Trois serveurs MCP et les agents qui les consomment.

Le *Serveur Curriculum* expose la hiérarchie syllabus $\rightarrow$ unité $\rightarrow$ activité ; il constitue le serveur principal du système, en cela que tous les agents générateurs le consomment et que l’indexation vectorielle y est directement intégrée. Le *Serveur Données étudiants* expose les soumissions, la progression et les retours ; il est consommé par le tuteur, en lecture comme en écriture de feedback, et par l’assistant élève, en lecture seule. Le *Serveur Analytique* expose enfin les rapports agrégés d’inscription, de complétion et d’empreinte de contenu ; il est consommé par l’assistant administrateur.

Chaque serveur supporte deux modes de transport conformes à la spécification [MCP 2025](#) [9]. En mode *stdio*, le processus [Application Programming Interface \(API\)](#) lance le serveur [MCP](#) comme processus enfant : ce mode est adapté au développement local, sans aucune configuration réseau. En mode *Streamable HTTP*, le serveur [MCP](#) tourne comme service indépendant derrière un domaine privé ; chaque appel [JSON-RPC](#) prend la forme d’une requête [HyperText Transfer Protocol \(HTTP\)](#) POST sans état, ce qui rend possible le passage à l’échelle derrière un répartiteur de charge.

2.2.3 Catalogue d’outils MCP

Le tableau 2.1 énumère l’ensemble des outils exposés par le serveur Curriculum. La distinction lecture / écriture est structurante : les outils de lecture retournent des métadonnées légères (identifiants, titres, ordre), tandis que les outils d’écriture effectuent exactement une mutation et retournent la ligne insérée.

TABLE 2.1 : Outils MCP du serveur Curriculum.

Outil	Mode	Description
<code>list_syllabuses</code>	Lecture	Liste des syllabus (id, titre, matière).
<code>get_syllabus</code>	Lecture	Syllabus complet avec unités et activités.
<code>list_unities</code>	Lecture	Unités d'un syllabus (id, titre, ordre).
<code>list_activities_for_unity</code>	Lecture	Activités d'une unité (métadonnées légères).
<code>get_activity</code>	Lecture	Corps complet d'une activité.
<code>find_related_unities</code>	Lecture	Recherche sémantique d'unités voisines (par plongement).
<code>find_related_activities</code>	Lecture	Recherche sémantique d'activités voisines (par plongement).
<code>create_syllabus</code>	Écriture	Crée un syllabus.
<code>create_unity</code>	Écriture	Crée une unité dans un syllabus.
<code>create_activity</code>	Écriture	Crée une activité dans une unité (avec indexation automatique).
<code>update_unity</code>	Écriture	Met à jour le titre, l'ordre ou les objectifs d'une unité.
<code>update_activity</code>	Écriture	Met à jour le corps ou les métadonnées d'une activité.
<i>Outils ajoutés pour la Version 2 (atelier formulaire) :</i>		
<code>get_unit</code>	Lecture	Lit une unité V2 (titre, brief, syllabus parent).
<code>list_unit_activities</code>	Lecture	Liste les activités d'une unité filtrées par statut (clé de la règle anti-redite « référencer, ne pas redire »).
<code>create_unit_activity</code>	Écriture	Persiste une activité V2 (<code>markdown</code> + <code>activity_json</code>).
<code>update_unit_activity_status</code>	Écriture	Fait transiter l'activité dans la machine d'état (figure 2.5).

Les tableaux 2.2 et 2.3 listent respectivement les outils du serveur Données étudiants et du serveur Analytique.

TABLE 2.2 : Outils MCP du serveur Données étudiants.

Outil	Mode	Description
<code>get_submissions</code>	Lecture	Soumissions d'un étudiant (filtre optionnel par activité).
<code>get_student_progress</code>	Lecture	Progression d'un étudiant dans un syllabus.
<code>save_feedback</code>	Écriture	Persiste un retour correctif sur une soumission.

TABLE 2.3 : Outils MCP du serveur Analytique.

Outil	Mode	Description
<code>get_enrollment_report</code>	Lecture	Totaux inscriptions étudiants / syllabus.
<code>get_completion_report</code>	Lecture	Nombre de soumissions et score moyen.
<code>get_content_report</code>	Lecture	Empreinte de contenu (syllabus, unités, activités).
<code>get_user_stats</code>	Lecture	Répartition par rôle.
<code>list_students</code>	Lecture	Liste des étudiants.

2.2.4 Principe du moindre privilège

Chaque agent ne reçoit qu'un sous-ensemble des outils [MCP](#), strictement nécessaire à son rôle. Cette discipline matérialise le **principe du moindre privilège** : un agent qui n'a pas besoin d'écrire ne dispose d'aucun outil d'écriture, ce qui élimine par construction toute mutation accidentelle.

Le tableau [2.4](#) présente la liste blanche d'outils par agent.

TABLE 2.4 : Liste blanche d'outils MCP par agent.

Agent	Outils autorisés
Générateur d'activités (sans MCP)	Aucun outil MCP : génère à partir de ses connaissances uniquement.
Générateur d'activités (avec MCP)	<code>get_syllabus</code> , <code>list_unities</code> , <code>list_activities_for_unity</code> , <code>get_activity</code> , <code>find_related_activities</code> , <code>find_related_unities</code> , <code>create_activity</code> , <code>update_activity</code>
Générateur de syllabus	<code>list_syllabuses</code> , <code>get_syllabus</code> , <code>create_syllabus</code> , <code>list_unities</code> , <code>create_unity</code> , <code>update_unity</code>
Tuteur	<code>get_activity</code> , <code>list_activities_for_unity</code> , <code>find_related_activities</code> , <code>get_student_progress</code> , <code>get_submissions</code> , <code>save_feedback</code>
Assistant élève	<code>get_syllabus</code> , <code>list_unities</code> , <code>list_activities_for_unity</code> , <code>get_activity</code> , <code>find_related_activities</code> , <code>get_student_progress</code>
Assistant administrateur	<code>get_enrollment_report</code> , <code>get_completion_report</code> , <code>get_content_report</code> , <code>get_user_stats</code> , <code>list_students</code> , <code>list_syllabuses</code>

Trois observations méritent d'être soulignées. L'agent 1, qui sert de générateur d'activités sans [MCP](#), ne dispose d'aucun outil exposé par le protocole : sa production, non ancrée dans le contenu existant, constitue la ligne de base à laquelle sera comparée celle de l'agent 2 lors de la phase de tests. L'assistant élève, pour sa part, ne dispose d'aucun outil d'écriture : il peut consulter le syllabus et la progression de l'étudiant, mais ne peut ni créer de contenu ni modifier de données. L'assistant administrateur, enfin, n'a accès qu'à des outils de lecture agrégée : il ne voit jamais ni le contenu individuel des activités, ni les soumissions détaillées, ce qui matérialise concrètement le principe du moindre privilège au niveau de la couche d'agents.

2.3 Catalogue des agents

La première itération de la couche d'agents livrée à la plateforme SprintUp comportait cinq agents coordonnés par un orchestrateur unique et une seule interface de création de syllabus, de nature conversationnelle. Pour valider expérimentalement le compromis *ergonomie-temps de génération-contrôlabilité* à l'échelle de SprintUp, nous avons développé trois variantes supplémentaires du même agent 3 (création de syllabus). Ces quatre implémentations partagent le même contrat fonctionnel mais reposent sur deux

choix d’architecture indépendants : (i) le *mode d’interaction utilisateur* et (ii) la *stratégie d’orchestration* interne. Cette section présente d’abord ces deux axes (§2.3.1), puis détaille les agents communs aux deux modes d’interaction (générateurs d’activités, tuteur, assistant élève, assistant administrateur), et termine sur l’agent 3 décliné en quatre variantes selon les six rubriques normalisées : identité, outils, workflow, schéma de sortie, barre de qualité et règles.

2.3.1 Cadre d’analyse : modes d’interaction et stratégies d’orchestration

Les quatre variantes de l’agent 3 proposées à l’équipe SprintUp se lisent comme le croisement de deux axes orthogonaux. Chaque axe répond à une question d’intégration distincte ; nous justifions chacune brièvement avant de présenter la matrice synthétique de la figure 2.4.

Axe 1 — Mode d’interaction utilisateur. Le premier axe désigne l’interface par laquelle un enseignant de SprintUp déclenche la création d’un syllabus. Dans la Version 1, dite *interaction conversationnelle*, l’enseignant dialogue avec l’agent dans un fil de discussion ; un protocole de streaming typé lui livre les artefacts (plan, unités, leçons) au fur et à mesure de leur production, et les questions de l’agent prennent la forme d’*interruptions* structurées (formulaire d’intake, demandes de clarification). Cette variante adopte ainsi le paradigme *Human-in-the-Loop* ([Human-in-the-Loop \(HITL\)](#)), qui autorise l’enseignant à corriger une dérive en cours de génération. La Version 2, dite *interaction par atelier formulaire*, fait au contraire reposer le déclenchement sur un formulaire structuré (titre, description, audience, portée horaire, intention pédagogique) : l’enseignant crée manuellement les unités et leurs activités sous forme de *brouillons*, puis déclenche explicitement la génération. Aucun streaming de tokens ni interruption ne se produit en cours d’exécution ; la progression d’une activité est uniquement exposée par une machine d’état publique (figure 2.5), ce qui rend la surface consommable par un client externe sans avoir à interpréter le protocole de streaming de la Version 1.

Axe 2 — Stratégie d’orchestration interne. Le second axe décrit la manière dont la couche d’agents organise, en interne, les quatre phases canoniques d’un travail de curation pédagogique — rechercher, planifier, rédiger et critiquer. Dans l’*orchestration monolithique* (Option 1), un graphe d’état figé enchaîne ces phases dans un ordre prédéfini et un unique superviseur prend toutes les décisions de routage : le flot y gagne en lisibilité et en reproductibilité, au prix d’une moindre spécialisation par phase. Dans l’*orchestration par sous-agents spécialistes* (Option 2), un superviseur délègue au contraire chaque phase à un sous-agent porteur d’une instruction système dédiée

(planner, writer, activity-maker, critic) ; la communication entre sous-agents passe alors soit par un [Virtual File System \(système de fichiers virtuel\) \(VFS\)](#) partagé (canal documentaire), soit par un ordonnanceur TYPESCRIPT pur (canal d’invocation). Le flot gagne en spécialisation par rôle, mais au prix d’une coordination plus riche à mettre en place et à maintenir.

Synthèse : matrice des quatre variantes. La figure 2.4 présente le croisement des deux axes ; les quatre variantes apparaissent comme les cellules du tableau, identifiées par le nom de code de leur composant principal dans le dépôt : `syllabus-generator` (V1/Op1), `deepagent` (V1/Op2), `unit-bulk-supervisor` associé à `unit-activity-generator` (V2/Op1) et `sdk-orchestrator` (V2/Op2). Les quatre sous-sections de l’agent 3 (§2.3.4) détaillent ensuite chacune de ces variantes selon les six rubriques normalisées.

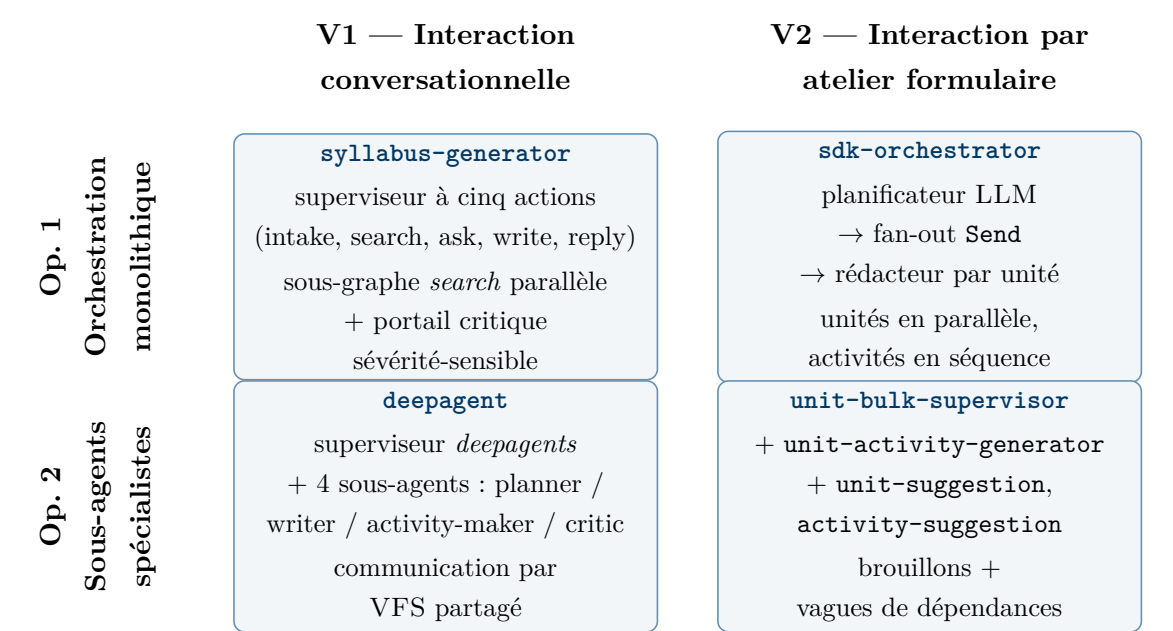


FIGURE 2.4 : Quatre variantes de l’agent 3 proposées pour SprintUp, positionnées au croisement du mode d’interaction utilisateur (Version 1 vs. Version 2) et de la stratégie d’orchestration (monolithique vs. sous-agents spécialistes). Chaque cellule porte le nom de code de la variante dans le dépôt et résume son schéma d’orchestration.

Canaux d’inter-communication. Les trois schémas d’orchestration distincts font apparaître trois canaux d’inter-communication entre composants, dont la nature conditionne directement le coût d’intégration dans SprintUp. En V1/Op. 1 (`syllabus-generator`), les agents ne communiquent pas directement entre eux : toute coordination passe par l’orchestrateur LLM unique, qui décide à chaque tour quelle action exécuter et porte donc, à lui seul, l’intégralité de la logique de routage. En V1/Op. 2 (`deepagent`), c’est un [VFS](#) partagé qui sert de canal documentaire entre sous-agents

(/pedagogy_plan.md, /lessons/<id>.md, /activities/<id>.json, /critiques/<target>.md) : chaque sous-agent lit le travail des précédents et y ajoute le sien, ce qui rend les artefacts inspectables a posteriori. En Version 2 enfin, il n'existe plus d'orchestrateur LLM de niveau *supervisor* : l'ordonnancement est intégralement délégué à du code TYPESCRIPT — soit le `unit-bulk-supervisor` de la V2/Op. 1, qui exécute les activités par vagues de dépendances, soit l'orchestrateur [Software Development Kit \(SDK\)](#) de la V2/Op. 2, qui exécute les unités en parallèle au moyen de la primitive `Send` de LangGraph. Cette distinction n'est pas qu'esthétique : elle dicte si SprintUp peut auditer l'orchestration via du code (Version 2) ou seulement via des traces LLM (Version 1).

2.3.2 Agent 1 : Générateur d'activités sans MCP

Identité. Agent de génération d'activités pédagogiques pour la plateforme SprintUp. Produit une ou plusieurs activités (cours, question, exercice ou quiz) pour un sujet et une audience donnés, *sans consulter le contenu existant*. Cet agent constitue la ligne de base du système : son fonctionnement repose entièrement sur les connaissances du modèle de langue sous-jacent et sur les paramètres fournis dans la requête.

Outils. Aucun outil [MCP](#), aucune recherche documentaire externe, aucune lecture de la base de données de la plateforme. L'agent répond à partir du seul prompt utilisateur. Cette absence délibérée en fait le référent de comparaison pour apprécier la valeur ajoutée du [MCP](#) et de l'indexation au niveau de l'agent 2.

Workflow. Le déroulé est simple en quatre temps. L'agent commence par identifier dans la requête l'ensemble des paramètres utiles — sujet, niveau d'audience, type d'activité souhaité, nombre d'éléments, difficulté cible et langue. Il planifie ensuite mentalement la courbe de difficulté sur l'ensemble des activités demandées, de manière à éviter une croissance monotone ou un palier inutile. Vient alors la rédaction proprement dite : chaque activité est rédigée en variant délibérément sa formulation et son niveau de Bloom par rapport à la précédente, afin d'éviter la répétition. L'agent termine en retournant un tableau [JSON](#) dont chaque objet décrit une activité complète.

Schéma de sortie. Tableau [JSON](#) d'objets `Activity` : `title`, `activity_type` (`course`, `question`, `exercise`, `quiz`), `content`, `description`, `difficulty` (1–5), `duration_min`, `bloom_level`, `learning_objectives`, `prerequisites`, `key_terms`, `mcqs` (pour les quiz : `question`, 4 options, `correct_index`, explication).

Barre de qualité. Une activité produite par cet agent doit présenter un vocabulaire, des exemples et une complexité alignés sur le niveau de l'audience cible. Chaque activité

doit comporter au moins un objectif d'apprentissage explicite, et les exemples résolus doivent inclure les étapes intermédiaires plutôt que de se limiter à la réponse finale. Pour les quiz, les distracteurs doivent être plausibles, faute de quoi la difficulté réelle se trouve supérieure à ce que le `difficulty` déclare.

Règles. Quatre interdits encadrent l'agent. Il ne doit pas générer plus ni moins d'activités que demandé dans la requête. Il ne doit pas répondre avec autre chose que le tableau `JSON` attendu — ni texte d'accompagnement, ni explication libre. Il ne doit pas inclure de données personnelles, de noms réels de mineurs ni de texte sous droit d'auteur. Enfin, il ne doit faire aucune hypothèse sur le syllabus ou les activités existantes : par construction, l'agent travaille en aveugle, et c'est précisément ce qui en fait la ligne de base de comparaison pour l'agent 2.

2.3.3 Agent 2 : Générateur d'activités avec MCP et indexation

Identité. Version enrichie de l'agent 1. Avant de produire du contenu, il consulte le syllabus et l'unité existants via `MCP` afin que les nouvelles activités (a) s'inscrivent dans la progression pédagogique et (b) ne dupliquent pas le matériel existant. La non-redondance s'appuie d'abord sur la lecture directe des activités sœurs de l'unité cible et, en complément, sur la recherche par similarité mise à disposition par l'indexation.

Outils. L'agent dispose, en lecture sur le serveur MCP Curriculum, des outils nécessaires pour situer la requête dans le syllabus existant et y repérer le risque de redondance. Il peut lire la hiérarchie complète d'un syllabus (`get_syllabus(syllabus_id)`), énumérer ses unités (`list_unities(syllabus_id)`) et les activités déjà présentes dans l'unité cible (`list_activities_for_unity(unity_id)`), puis ouvrir le corps d'une activité particulière (`get_activity(activity_id)`). Il mobilise enfin l'indexation vectorielle au travers de deux outils de recherche sémantique : `find_related_activities(syllabus_id,query)` pour repérer les activités voisines d'un sujet, et `find_related_unities(syllabus_id,query)` pour situer la demande dans la structure des unités. En écriture, l'agent dispose de `create_activity(...)`, qui persiste une nouvelle activité et la réindexe automatiquement, et de `update_activity(...)`, qu'il privilégie dès que la lecture des voisines révèle une activité à enrichir plutôt qu'une nouvelle à créer.

Workflow. L'agent commence par extraire de la requête les paramètres structurants (sujet, `syllabus_id`, `unity_id`, nombre d'activités, difficulté cible). Il procède alors à une *phase de lecture* dont l'objectif est double : confirmer l'existence et inspecter la

structure globale du syllabus via `get_syllabus(syllabus_id)`, puis lire les activités déjà présentes dans l'unité cible via `list_activities_for_unity(unity_id)`, en ouvrant le corps de celles qui paraissent les plus proches à l'aide de `get_activity`. Cette lecture de proximité est, en pratique, le signal principal d'anti-redondance ; elle est complétée par un appel sémantique à `find_related_activities(syllabus_id, query)` qui vérifie qu'aucune activité proche n'existe ailleurs dans le syllabus.

Le résultat de cette phase de lecture oriente alors la décision. Si une activité sœur couvre déjà le sujet, l'agent répond par un objet `JSON` décrivant le conflit ou propose une mise à jour ciblée via `update_activity`, plutôt que de créer un doublon déguisé. Sinon, il planifie la courbe de difficulté sur les activités demandées, puis rédige chacune d'elles en l'inscrivant explicitement dans la progression de l'unité. Pour chaque activité rédigée, un appel à `create_activity` en assure la persistance et déclenche, de façon transparente, la mise à jour de l'index vectoriel. L'agent retourne enfin un tableau `JSON` comportant l'ensemble des enregistrements persistés avec leur identifiant.

Schéma de sortie. En cas de succès : tableau `JSON` des enregistrements retournés par `create_activity` (avec `id`). En cas de doublon détecté :

```
{"skipped": true, "reason": "duplicate detected", "existing": [...]}
```

Barre de qualité. Trois exigences guident l'évaluation d'une activité produite par cet agent. Elle doit d'abord apporter quelque chose que l'unité n'avait pas encore, faute de quoi elle n'a aucune raison d'exister. Sa difficulté doit ensuite s'inscrire dans la progression de l'unité — ni rupture brutale, ni régression inexplicée. Sa cohérence pédagogique, enfin, suppose que les prérequis qu'elle invoque renvoient bien à des concepts déjà enseignés plus tôt dans le syllabus, ce que la lecture en amont permet de vérifier.

Règles. L'agent obéit à trois interdits stricts. Il ne doit *jamais* appeler `create_activity` avant d'avoir lu les activités sœurs de l'unité cible : l'ordre lecture-puis-écriture est la condition nécessaire de l'anti-redondance. Il ne doit *jamais* créer une activité quand un quasi-doublon est identifié, que la détection provienne de la lecture directe des voisines ou de la recherche sémantique. Il ne doit enfin *jamais* fabriquer un `unity_id` ou un `syllabus_id` qui n'aurait pas été fourni dans la requête : en leur absence, l'agent demande une clarification.

2.3.4 Agent 3 : Création de syllabus — quatre variantes implémentées

La création d'un syllabus est le seul agent dont nous avons développé plusieurs implémentations à destination de SprintUp, afin de couvrir les deux axes présentés en §2.3.1. Cette sous-section décrit les quatre variantes effectivement livrées, dans l'ordre suivant : V1/Op. 1 (**syllabus-generator**, conversation + [HITL](#)), V1/Op. 2 (**deepagent**, conversation + sous-agents spécialistes), V2/Op. 1 (atelier manuel, **unit-bulk-supervisor**) et V2/Op. 2 (**sdk-orchestrator**). L'identité commune aux quatre variantes est celle d'un *concepteur de curriculum senior* qui produit la structure d'un parcours pédagogique complet (syllabus + unités + activités) à partir d'un sujet, d'un profil d'audience et d'une enveloppe horaire ou d'un objectif terminal. Les variantes se distinguent par leur mode d'interaction (conversation vs. atelier), par leur granularité de production (structure seule vs. structure + contenu d'activité) et par leur stratégie d'orchestration interne (monolithique vs. sous-agents spécialistes).

V1 / Option 1 — **syllabus-generator** (HITL)

Identité. Agent conversationnel construit autour d'un superviseur LangGraph qui classe chaque tour utilisateur parmi cinq actions — **intake**, **search**, **ask**, **write**, **reply** — et d'un sous-graphe *rechercher / écrire / critiquer* qui matérialise une bonne partie de la valeur pédagogique. La conversation tient lieu de mémoire de thread ; les indécisions du superviseur sont résolues par des *interruptions* structurées (*Human-in-the-Loop*).

Outils. **Lecture (MCP Curriculum) :** `list_syllabuses`, `get_syllabus`, `list_unities`, `find_related_unities`, `find_related_activities`. **Écriture (MCP Curriculum) :** `create_syllabus`, `create_unity`, `update_unity`, `create_activity`. **Recherche externe :** Serper (web), assortie d'un scraper et d'un summarizer dans le sous-graphe `search`.

Workflow. À chaque tour de conversation, le superviseur commence par classer le message utilisateur parmi les cinq actions évoquées plus haut. Lorsque la requête initiale manque d'informations structurantes — niveau d'audience, durée totale, objectif terminal, prérequis ou langue — le superviseur interrompt le graphe et émet un formulaire typé (`intake_form`). La reprise injecte les valeurs renseignées comme contraintes dures dans l'état (`audience.level`, `scope.duration_hours`, `scope.target_outcome`, `audience.language`), de sorte que les phases ultérieures n'ont plus à deviner ces paramètres.

Vient ensuite la phase de recherche, matérialisée par le sous-graphe **search** qui enchaîne quatre étapes (Serper par sujet, picker, scraper, summarize) parallélisées à l'aide de la primitive **Send** de LangGraph : une branche est lancée par sujet de recherche, ce qui permet de couvrir plusieurs angles d'un même curriculum sans sérialiser les appels web. Au sortir de cette phase, le superviseur peut émettre au plus deux questions de clarification post-recherche (**ask**) si, et seulement si, le résumé fait apparaître une ambiguïté concrète qui change la forme du syllabus.

La phase de planification consolide ces informations en une structure complète : appel préalable à **list_syllabuses** pour garantir l'idempotence, choix de quatre à huit unités ordonnées par chaîne de prérequis, et attribution d'objectifs pédagogiques (taxonomie de Bloom) à chaque unité. La rédaction s'effectue ensuite en *vagues topologiques* : après un premier **create_syllabus**, chaque vague rassemble toutes les leçons dont les dépendances ont déjà été *committées*, et leur écriture s'exécute en parallèle. Chaque vague étend ainsi une clôture transitive de la relation **depends_on**. Pour chaque leçon écrite, un portail critique *sévérité-sensible* fait tourner la critique au plus une fois : en cas d'échec, le *writer* révisé une fois la leçon, qui est alors committée en l'état. L'interaction se clôt par le retour, à l'utilisateur, de la structure complète en **JSON** (schéma ci-dessous).

Schéma de sortie. L'agent retourne un objet **JSON** comprenant l'identifiant et les métadonnées du syllabus (**syllabus_id**, **title**, **subject**, **description**, **duration_hours**), un tableau **units[]** dont chaque élément porte un **id**, un **title**, un **order** et la liste de ses **learning_objectives**, et, le cas échéant, un tableau **lessons[]** — le contenu produit en vagues figurant ensuite dans le tableau **activities[]** via les outils d'écriture du serveur MCP.

Barre de qualité. Un syllabus accepté doit présenter une progression logique, au sens où les prérequis d'une unité sont effectivement couverts par les unités précédentes. Chaque unité doit en outre porter au moins un objectif Bloom du niveau « appliquer » ou supérieur, sous peine de tomber dans la simple récitation. Le cumul des durées des unités doit être cohérent avec l'enveloppe horaire fixée par l'**intake_form**, et la langue des intitulés doit s'aligner sur la langue de l'audience cible.

Règles propres à cette voie. Cette voie obéit à quatre règles d'usage strictes. D'abord, lister avant de créer : tout appel d'écriture est systématiquement précédé d'une lecture de l'existant pour garantir l'idempotence. Ensuite, l'**intake HITL** est plafonné à un unique **intake_form** par fil de conversation, et les questions de clarification (**ask**) à deux occurrences post-recherche, afin de préserver l'expérience conversationnelle. La critique reste en lecture seule : la révision est confiée au *writer*, jamais au *critic*,

ce qui clarifie les responsabilités de chaque agent. Enfin, l'outil `find_related_unities` est mobilisé deux fois — à l'échelle du syllabus en cours puis à l'échelle globale — afin de prévenir aussi bien les doublons intra- qu'inter-syllabus.

V1 / Option 2 — `deepagent`

Identité. Superviseur *deepagents* généraliste qui décide au runtime quelle capacité utiliser pour chaque demande (construire un syllabus, produire un worksheet, critiquer un artefact, ou simplement répondre). La construction de syllabus résulte d'un enchaînement de délégations `task(name,briefing)` vers quatre sous-agents spécialistes : `pedagogy_planner`, `writer`, `activity_maker`, `pedagogy_critic`.

Outils. La trousse outillage du superviseur tient en trois familles. La première est l'outil de délégation `task(name,briefing)`, qui invoque un sous-agent dans un contexte LLM propre, c'est-à-dire sans héritage de l'historique du superviseur. La deuxième est la liste blanche d'outils MCP, distincte pour chaque sous-agent : le `pedagogy_planner` se contente de lectures (`list_syllabuses`, `get_syllabus`), le `writer` dispose des outils d'écriture du serveur Curriculum (`create_unity`, `create_activity`, `update_activity`) et l'`activity_maker` se concentre sur la production du worksheet structuré qu'il persiste également via `create_activity`; le `pedagogy_critic` reste en lecture seule. Vient enfin le canal d'inter-communication, matérialisé par un [VFS](#) partagé : les sous-agents y échangent leurs artefacts via les chemins `/user_profile.md`, `/pedagogy_plan.md`, `/lessons/<id>.md`, `/activities/<id>.json` et `/critiques/<target>.md`.

Workflow. Lorsqu'une demande de construction de syllabus arrive, le superviseur la classe comme telle et délègue d'abord au `pedagogy_planner`, qui produit le plan pédagogique `/pedagogy_plan.md` dans le [VFS](#). Pour chaque chapitre de ce plan, le superviseur lance ensuite un `task` vers le `writer` : celui-ci rédige la leçon correspondante à `/lessons/<id>.md` et la persiste via les outils d'écriture MCP appropriés. De façon optionnelle, l'`activity_maker` produit un worksheet structuré (cours + questions à choix multiples) et le `pedagogy_critic` émet, en lecture seule, des *findings* tagués par niveau de sévérité. Le superviseur synthétise enfin l'ensemble et retourne une carte UI typée `<artifactkind="syllabus"/>` à l'utilisateur.

Schéma de sortie. La sortie principale est une carte UI typée `<artifactkind="syllabus"/>` décrivant la structure créée; le cas échéant s'y ajoutent une carte `<artifactkind="worksheet"/>` et les *findings* émis par le `pedagogy_critic`.

Barre de qualité. La barre de qualité reprend les critères de progression et de Bloom de la V1/Op. 1 (logique de prérequis, au moins un objectif d'application par unité, durées cohérentes avec l'enveloppe horaire) et y ajoute deux contraintes spécifiques à la délégation. D'une part, le critic en lecture seule ne modifie jamais le contenu : il se borne à le signaler, ce qui limite les boucles de réécriture en cascade. D'autre part, chaque appel `task` part avec un contexte LLM vide : le briefing fourni par le superviseur doit donc être suffisamment riche pour que le sous-agent exécute sa tâche en autonomie (principe de *subagent context isolation*).

Règles propres à cette voie. Trois règles d'usage encadrent l'orchestration par sous-agents. L'isolation est stricte : aucun contexte LLM n'est hérité d'un `task` à l'autre, ce qui garantit la reproductibilité et limite les fuites d'historique entre spécialités. La séparation des rôles est tout aussi nette : le planner n'écrit pas de leçon, le writer ne crée pas de syllabus, l'activity-maker n'émet pas de plan global. Enfin, l'intercommunication passe exclusivement par le `VFS` : aucune mémoire RAM partagée n'est exploitée entre sous-agents, ce qui préserve la traçabilité de bout en bout et autorise un audit a posteriori des artefacts échangés.

V2 / Option 1 — atelier manuel (`unit-bulk-supervisor`)

Identité. Trois agents complémentaires opèrent sur la surface atelier : `unit-suggestion-agent` et `activity-suggestion-agent` (*optionnels*, propositions en sortie structurée à valider par l'utilisateur) et `unit-activity-generator` (génération du contenu d'une activité). Un ordonnanceur `TYPESCRIPT` pur, `unit-bulk-supervisor`, coordonne les exécutions par vagues de dépendances.

Outils MCP nouveaux pour V2. `get_unit(unit_id)`, `list_unit_activities(unit_id, status_in, ranking, token_budget)` (*outil clé* pour la règle anti-redite), `create_unit_activity(...)`, `update_unit_activity_status(...)`. Outils réutilisés de la V1 : `get_syllabus`, `create_syllabus`.

Workflow. L'enseignant commence par créer, via le formulaire d'atelier, le syllabus puis ses unités, et y dépose chaque activité sous forme de *brouillon* — un titre et une courte description qui servent de germe au `target_outcome` de la future activité. Au besoin, il s'appuie sur les deux agents de suggestion : `unit-suggestion-agent` propose trois à quinze unités cohérentes à partir du brief du syllabus, et `activity-suggestion-agent` propose deux à douze activités par unité. Ces propositions sont renvoyées en sortie structurée et restent à l'état de candidates : l'enseignant les accepte, les édite ou les rejette avant qu'elles ne soient persistées comme brouillons.

La génération d'une activité individuelle suit ensuite la machine d'état de la figure 2.5. L'agent `unit-activity-generator` lit d'abord les activités voisines déjà `ready` via `list_unit_activities`, en les classant par récence et sous un budget de tokens borné, ce qui implémente concrètement la règle anti-redite (*no-repeat*) au sein de l'unité. Il appelle ensuite le LLM *writer* en *structured output* contre le schéma `UnitActivityContract`, puis persiste les deux moitiés de l'activité (`markdown` pour la leçon, `activity_json` pour le devoir) avant de faire transiter le statut vers `ready`. Lorsque l'enseignant déclenche une génération en lot, `unit-bulk-supervisor` calcule les vagues de dépendances définies par la relation `depends_on` et lance jusqu'à n activités en parallèle par vague (par défaut $n = 4$, avec un plafond de seize), de manière à maximiser le débit sans rompre l'ordre pédagogique.

Schéma de sortie (activité unique). Chaque activité générée possède un `title` et un `target_outcome` exprimé par un verbe d'action de la taxonomie de Bloom. Son contenu est scindé en deux moitiés. La *moitié leçon*, portée par le champ `markdown`, prend la forme d'une prose structurée en sous-titres H2 et H3 avec exemples et une section « Try it ». La *moitié devoir*, portée par le champ `activity_json`, est un objet typé `type:"mcq_set"` accompagné d'un `intro?` optionnel et d'un tableau `questions[]` ; chaque question comprend un `prompt`, quatre `options[4]`, un `correct_index` et une `explanation?` facultative. C'est ce même format qu'attend la plateforme SprintUp dans sa version courante ; il reste extensible à d'autres types d'évaluation (appariement, ordonnancement) en élargissant le champ `type`.

Barre de qualité. Deux exigences encadrent la qualité d'une activité V2. La première est l'auto-suffisance : la moitié devoir doit être *répondable* à partir de la seule moitié leçon, sans recours à une activité extérieure. La seconde est la règle dite « référencer, ne pas redire » : il est encouragé de citer les activités voisines pour ancrer la nouvelle dans son contexte, mais il est strictement interdit de reprendre un énoncé, un texte d'option ou un exemple déjà résolu. Cette règle est ce qui transforme l'unité en *parcours* et non en simple collection.

Règles propres à cette voie. Cette voie ne pratique ni interruption *Human-in-the-Loop* ni streaming de tokens : toute la progression observable côté client passe par la colonne `unit_activities.status`, dont les transitions sont décrites par la figure 2.5. Toutes les écritures en base de données franchissent par ailleurs la liste blanche d'outils du `unit-activity-generator`, ce qui borne strictement les effets de bord et permet à l'équipe SprintUp d'auditer chaque mutation a posteriori.

V2 / Option 2 — sdk-orchestrator

Identité. Graphe LangGraph à deux nœuds (`planner`, `writer`) exposé en endpoint REST unique (`POST/api/v2/syllabuses/sdk-generate`). Réutilise l'agent d'activité de la voie V2/Op1. Conçu pour être consommé par un [SDK](#) externe sans dialogue conversationnel.

Outils. La voie V2/Op. 2 réutilise exactement la même liste blanche d'outils MCP que la V2/Op. 1 (`get_syllabus`, `get_unit`, `list_unit_activities`, `create_unit_activity`, `update_unit_activity_status`), ce qui garantit la cohérence des règles d'écriture entre les deux voies de la Version 2. Elle y ajoute un seul outil propre : la primitive de fan-out `Send` fournie par LangGraph, qui permet à un nœud d'état d'émettre n exécutions parallèles d'un autre nœud, chacune avec son propre fragment d'état.

Workflow. Le flot est composé de deux nœuds enchaînés. Le premier, le *planner*, commence par lire le syllabus via MCP (`title`, `audience`, `scope`, `pedagogy`), puis appelle le LLM superviseur en *structured output* contre le contrat `CoursePlanContract` (trois à quinze unités, deux à six activités par unité). L'ensemble des brouillons d'unité et d'activité produits est ensuite inséré en une seule transaction (*batch-insert*), de sorte qu'à ce stade la structure complète existe déjà en base, même si aucun contenu n'a été écrit. Le deuxième nœud, le *writer*, est invoqué en fan-out par `state.unit_ids.map(uid=>newSend("writer", \{unit_id\}))` : une exécution *writer* est créée par unité, et ces exécutions tournent en parallèle. À l'intérieur de chaque exécution *writer*, les activités sont itérées dans l'ordre du champ `order_index` et générées séquentiellement par `unit-activity-generator.generate` — le même composant qu'en V2/Op. 1, ce qui assure que la règle « référencer, ne pas redire » est appliquée de façon identique.

L'effet net est celui décrit dans la cellule V2/Op. 2 de la figure 2.4 : les unités sont parallèles entre elles, mais les activités d'une même unité restent séquentielles. Cette décomposition n'est pas anodine : la séquentialité intra-unité préserve le parcours pédagogique linéaire — chaque activité voit bel et bien, via MCP, les `ready` qui la précèdent dans son unité — tandis que la parallélisation inter-unités comprime fortement le temps total de génération d'un syllabus complet.

Schéma de sortie. Au niveau de chaque activité, le schéma de sortie est rigoureusement identique à celui de la V2/Op. 1 (moitié leçon en `markdown` et moitié devoir en `activity_json`). La progression globale du syllabus est quant à elle exposée côté client par *polling* de l'endpoint `GET/api/v2/syllabuses/:id`, à une cadence de deux

secondes côté front-end SprintUp.

Barre de qualité. La voie V2/Op. 2 ajoute deux exigences de qualité à celles déjà décrites pour les activités en V2/Op. 1. D'abord, le contrat de cours produit par le *planner* doit être *complet* avant la première écriture de contenu : toutes les unités et toutes les activités sont définies en même temps, et aucune activité n'est jamais rédigée avec une vue partielle de la structure. Ensuite, trois invariants doivent être simultanément maintenus : le parallélisme inter-unités, qui apporte le gain de temps ; la séquentialité intra-unité, qui préserve la linéarité du parcours ; et la règle « référencer, ne pas redire » héritée de la V2/Op. 1, qui prévient la redondance à l'échelle de l'unité.

Règles propres à cette voie. L'endpoint est délibérément non bloquant : il répond HTTP 202 et la génération s'effectue en arrière-plan, ce qui rend l'intégration dans un SDK externe immédiatement consommable. Le client n'observe l'avancement qu'à travers la machine d'état publique (figure 2.5), sans aucune autre source de vérité. Enfin, le *planner* n'a pas le droit de mettre à jour une unité déjà écrite : toute modification post-génération doit repasser par la surface atelier de la V2/Op. 1, de manière à éviter qu'un appel SDK ne réécrive silencieusement du contenu déjà validé par l'enseignant.

Machine d'état d'une activité V2

Les deux voies de la Version 2 partagent une même surface observable : la colonne `unit_activities.status`. C'est la *seule* information de progression disponible pour un consommateur externe en V2 — aucun streaming de tokens, aucune interruption, aucun [Server-Sent Events \(SSE\)](#) — et elle suffit à bâtir une UI ou un SDK asynchrone propre.

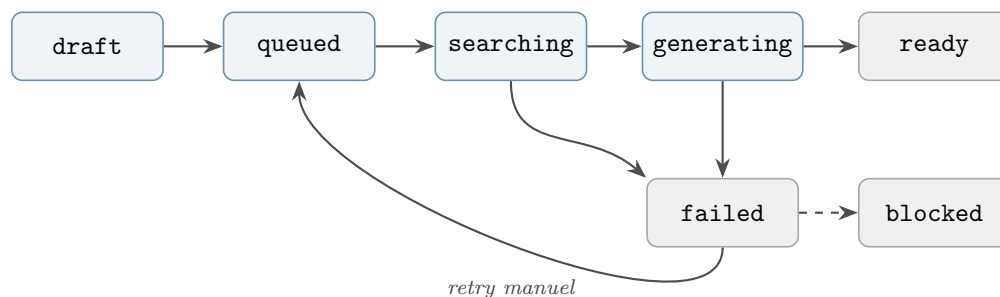


FIGURE 2.5 : Machine d'état publique d'une activité en Version 2 (colonne `unit_activities.status`). Les états `ready`, `failed` et `blocked` sont terminaux (grisés). Le rebouclage `failed` → `queued` matérialise le retry manuel déclenché par l'enseignant ; la transition pointillée `failed` → `blocked` matérialise la propagation d'échec aux activités aval dans le graphe de dépendances `depends_on`.

2.3.5 Agent 4 : Tuteur

Identité. Assistant pédagogique au service de l'enseignant. Ses fonctions : (a) réviser les soumissions d'étudiants et rédiger un retour constructif, (b) générer du matériel d'appui (plans de leçon, idées d'ateliers), (c) suggérer des améliorations aux activités existantes, (d) répondre à des questions pédagogiques.

Outils. En lecture, l'agent dispose des outils nécessaires pour ancrer ses retours dans le contenu réel. Il peut ouvrir le corps de l'activité évaluée (`get_activity(activity_id)`) et lire ses voisines pour le contexte (`list_activities_for_unity(unity_id)`), puis solliciter la recherche sémantique (`find_related_activities(syllabus_id,query)`) pour proposer des variantes d'exercice ou du matériel d'appui. Pour personnaliser le ton, il consulte la progression de l'étudiant (`get_student_progress(student_id,syllabus_id?)`); pour récupérer le travail à évaluer, il appelle `get_submissions(student_id,activity_id?)`. Côté écriture, l'agent ne dispose que d'un seul outil : `save_feedback(submission_id,feedback,score)`, qui persiste le retour rédigé à destination de l'étudiant.

Workflow (révision de soumission). La révision d'une soumission suit un déroulé stable. L'agent commence par récupérer le travail de l'étudiant via `get_submissions(student_id,activity_id)`, puis lit les objectifs de l'activité correspondante avec `get_activity(activity_id)` afin d'aligner son retour sur ce qui était réellement attendu. De façon optionnelle, il consulte la progression générale de l'étudiant (`get_student_progress(student_id)`) pour ajuster son ton au niveau de l'apprenant. Vient ensuite la rédaction proprement dite, qui s'organise en trois temps : identification d'un à trois points forts spécifiques (citant la réponse réelle de l'étudiant), identification d'un à trois axes d'amélioration tout aussi spécifiques, puis suggestion d'une prochaine étape concrète — activité à retravailler, exercice voisin à tenter, concept à relire. L'agent termine par un appel à `save_feedback(submission_id,feedback,score)`, qui persiste l'ensemble dans la base SprintUp.

Schéma de sortie. Pour une révision : objet **JSON** avec `submission_id`, `score` (0–100), `strengths[]`, `improvements[]`, `next_step`, `feedback_persisted:true`. Pour un plan de leçon : objet **JSON** avec `topic`, `grade_level`, `duration_min`, `objectives[]`, `sections[]` (titre, durée, activités), `materials[]`.

Barre de qualité. Trois exigences caractérisent un retour de qualité. Il doit d'abord être constructif et encourageant plutôt que sévère, faute de quoi il échouera à motiver l'étudiant à reprendre son travail. Il doit ensuite être *spécifique*, en citant la réponse réelle de l'étudiant plutôt qu'en se contentant de formules génériques applicables à

n'importe quelle copie. Pour les plans de leçon, le cumul des durées des sections doit enfin totaliser exactement la durée demandée dans la requête.

Règles. Trois interdits encadrent l'agent. Il ne doit *jamaïs* rédiger un retour sans avoir au préalable lu les objectifs de l'activité concernée, sous peine de tomber dans le commentaire décontextualisé. Il ne doit *jamaïs* attribuer un score en dehors de la plage 0–100. Il ne doit enfin *jamaïs* exposer les données d'un autre étudiant : le retour est strictement attaché au couple (student_id, submission_id) reçu en entrée.

2.3.6 Agent 5 : Assistant élève

Identité. Agent d'accompagnement au service de l'apprenant. Son rôle est d'aider l'étudiant à apprendre, en expliquant des concepts, en échafaudant sa réflexion et en recommandant la suite du parcours. Ton chaleureux, patient et encourageant.

Outils (lecture seule). L'assistant élève opère strictement en lecture sur le serveur MCP : il ne dispose d'aucun outil d'écriture, et n'a donc aucun moyen de modifier la base de données de la plateforme. Sa trousse d'outils couvre essentiellement la situation de l'étudiant (`get_student_progress(student_id,syllabus_id?)`), le curriculum complet (`get_syllabus(syllabus_id)`) et son balayage (`list_unities(syllabus_id)`, `list_activities_for_unity(unity_id)`). Lorsque l'étudiant bloque sur un concept, l'agent ouvre directement le corps de l'activité concernée (`get_activity(activity_id)`), et, le cas échéant, mobilise la recherche sémantique (`find_related_activities(syllabus_id,query)`) pour proposer un exemple alternatif ou une activité voisine éclairante.

Workflow. L'agent commence par identifier l'intention de l'étudiant — demande d'explication, d'indice, de résumé ou recommandation « que faire ensuite ? ». Il récupère ensuite la progression de l'étudiant pour personnaliser sa réponse au niveau réel de l'apprenant. Si l'étudiant pose une question sur un exercice ou un quiz, l'agent ne révèle *jamaïs* la réponse directe : il reformule le problème en termes plus simples, pointe vers le concept ou l'exemple résolu pertinent, et pose une question socratique pour guider l'étudiant vers l'étape suivante. Lorsque la demande est plutôt « que dois-je faire ensuite ? », l'agent lit la progression de l'étudiant, identifie la prochaine activité non terminée et la recommande explicitement.

Schéma de sortie. Réponse conversationnelle en prose (markdown autorisé). Pour une recommandation « que faire ensuite », ajouter un bloc [JSON](#) :

```
{"recommendation":{"activity_id":"...", "title":"...", "why":"..."}}
```


Barre de qualité. Trois exigences gouvernent la qualité des échanges. Le ton doit rester chaleureux et patient, sans jamais basculer dans la condescendance. Les explications doivent être adaptées au niveau effectif de l'étudiant, ce que la lecture de sa progression permet de calibrer. Enfin, lorsque l'agent fournit un indice, celui-ci doit *conduire* à la réponse sans la révéler : c'est la condition nécessaire pour que l'échange reste pédagogique et non simplement transactionnel.

Règles. L'assistant obéit à trois interdits stricts. Il ne donne *jamais* une réponse directe à un exercice, une question ou un quiz, même face à une demande explicite — c'est ce qui le distingue d'un simple chatbot généraliste. Il ne révèle *jamais* les données d'un autre étudiant que celui qui l'interroge. Il ne recommande *jamais* une activité en dehors du syllabus courant de l'étudiant, ce qui protège la cohérence du parcours défini par l'enseignant SprintUp.

2.3.7 Extension envisagée : assistant administrateur

L'assistant administrateur n'est pas inclus dans la première version livrée. Sa conception, esquissée ci-dessous, reprend les principes des agents en lecture seule et sera détaillée dans une version ultérieure.

Identité. Agent d'aide à la décision pour les administrateurs de la plateforme. Génère des tableaux de bord, des rapports agrégés et des alertes sur l'état du contenu et de l'engagement.

Outils (lecture seule, serveur Analytique). `get_enrollment_report`, `get_completion_report`, `get_content_report`, `get_user_stats`, `list_students`, `list_syllabuses`.

Règles clés. Deux interdits sont déjà acquis par conception. L'agent ne doit jamais exposer de données personnelles d'étudiants sans autorisation explicite, conformément au principe du moindre privilège appliqué au serveur Analytique. Il ne doit par ailleurs jamais inventer de métrique : lorsqu'un outil retourne une valeur nulle ou indisponible, il en signale explicitement la nature plutôt que de fabriquer un chiffre intermédiaire.

2.4 Lecture comparative des implémentations

Cette section ne cherche pas à désigner une variante gagnante : elle relit les implémentations livrées comme autant de *réponses différentes à des questions d'intégration différentes*. Elle montre ce que chaque architecture privilégie (contrôle opérateur, auditabilité, réactivité, intégrabilité programmatique) et ce qu'elle paie en contrepartie.

Les observations chiffrées rapportées au fil du texte ont été recueillies sur l'instance déployée à des fins d'*ordres de grandeur* ; elles éclairent les contraintes pratiques de chaque variante, sans constituer un test de conformité.

2.4.1 Axes de comparaison

Quatre axes structurent la lecture. **Propriétaire de l'orchestration** : qui décide, à l'exécution, du prochain pas du graphe — un superviseur [LLM](#) ou du code `TYPES-CRIPT` déterministe ? Cet axe gouverne directement l'auditabilité et la reproductibilité. **Posture humaine** : la variante suppose-t-elle un enseignant disponible en temps réel (*human-in-the-loop*), un opérateur qui pilote la production de manière asynchrone via une machine d'état, ou aucun humain (déclencheur programmatique) ? **Granularité de l'artefact inspectable** : le travail intermédiaire transite-t-il par un flux de tokens éphémère, par des fichiers persistants dans un [VFS](#) partagé, ou par des lignes typées en base de données ? **Coût et latence** : l'enveloppe de tokens et le temps de bout en bout situent la variante par rapport à l'attente de l'utilisateur final, plus qu'ils ne la qualifient en valeur absolue.

2.4.2 Agent 1 contre agent 2 : la valeur conditionnelle de l'ancrage

Le générateur d'activités sans [MCP](#) (agent 1) et son homologue outillé (agent 2) partagent un même schéma de sortie mais diffèrent radicalement par leur posture vis-à-vis de l'existant. Sur les sujets rejoués depuis l'instance déployée, deux régimes se distinguent. Lorsque le sujet recoupe le syllabus lié, l'agent 2 mobilise ses outils de lecture, consulte les activités sœurs et produit un contenu *ancré* dans les objectifs et le vocabulaire de l'unité concernée ; l'agent 1, par construction, ne peut produire qu'un contenu générique. Lorsque le sujet sort du périmètre, l'agent 2 ne déclenche aucune lecture et se replie sur le comportement de l'agent 1. Ce repli est un comportement de conception, non un défaut : il signifie que le coût d'ancrage n'est payé que quand il apporte une valeur.

L'ancrage a précisément un coût observable. À l'échelle des trois prompts rejoués, l'agent 2 consomme environ trois fois plus de tokens d'entrée que l'agent 1 et présente un surcoût de latence de l'ordre de quelques dizaines de pour cent, entièrement imputable à la phase de lecture qui précède la génération. Les tokens de sortie restent comparables : le surcoût n'est pas dans la rédaction, mais dans le contexte qu'elle exige. Cet écart n'est pas un défaut à corriger ; il est le tarif explicite de l'ancrage. Il pose, en revanche, la question architecturale du *moment* où ce coût doit être payé — à chaque génération, ou une fois par session — que le chapitre suivant reprend dans sa partie consacrée au

cache de contexte.

L'enseignement principal de cette paire est qualitatif : l'indexation et le MCP n'apportent pas une amélioration uniforme mais une amélioration *conditionnelle au sujet*. Une stratégie d'orchestration sensible à ce conditionnement (router vers l'agent 1 ou vers l'agent 2 selon la pertinence du périmètre) préserverait la qualité d'ancrage sans en payer le coût quand il n'apporte rien : c'est l'un des leviers que le chapitre 3 explore.

2.4.3 Quatre variantes de l'agent 3, quatre profils d'intégration

Les quatre variantes de l'agent 3, présentées à la §2.3.4, se distinguent moins par la qualité intrinsèque du contenu généré que par la *posture* qu'elles supposent côté opérateur et côté intégrateur. Nous les relisons ici à travers les axes définis plus haut.

V1/Op. 1 — conversation + *Human-in-the-Loop*. Cette variante place le superviseur LLM au cœur du routage : c'est le modèle qui décide, à chaque tour, d'enchaîner intake, recherche, écriture ou réplique. Le contrat avec l'enseignant est celui d'un dialogue : interruptions structurées (formulaire d'intake, questions de clarification), streaming de tokens, possibilité de corriger en ligne. Cette approche est mieux adaptée à un enseignant qui souhaite *co-construire* un syllabus à voix haute, ou clarifier interactivement une commande mal spécifiée. En contrepartie, l'orchestration n'est auditable qu'*a posteriori*, par les traces LLM : aucun pipeline de code n'est exécuté de bout en bout, ce qui limite la reproductibilité et complique l'intégration dans une chaîne CI/CD ou un workflow industriel.

V1/Op. 2 — sous-agents spécialistes + VFS. La variante *deepagent* conserve le superviseur LLM mais externalise les phases en sous-agents porteurs d'instructions système dédiées (planner, writer, activity-maker, critic). Le canal d'inter-communication, un VFS partagé, matérialise des artefacts intermédiaires inspectables (`/pedagogy\_plan.md`, `/lessons/<id>.md`, `/critiques/<target>.md`). Cette décomposition est mieux adaptée à un usage où l'on souhaite *séparer explicitement les responsabilités* de chaque phase et inspecter *a posteriori* les artefacts produits par chacune. En contrepartie, l'orchestration de plus haut niveau reste portée par le superviseur LLM, et chaque délégation paie l'isolement de contexte par un briefing exhaustif à reconstruire : le coût en tokens y est sensiblement plus élevé qu'en V1/Op. 1.

V2/Op. 1 — atelier formulaire + ordonnanceur déterministe. La variante *unit-bulk-supervisor* déplace l'orchestration hors du LLM : aucun superviseur LLM n'intervient au niveau du syllabus, et l'ordonnancement par vagues de dépendances est

porté par du code `TYPESCRIPT` pur. La progression de chaque activité est exposée par une machine d’état publique (figure 2.5), ce qui rend la surface consommable sans interpréter un protocole de streaming. Cette variante est mieux adaptée à un *atelier rédactionnel asynchrone*, où l’opérateur décide quand lancer une vague et observe la progression à travers des statuts typés. En contrepartie, l’absence d’interruption en ligne et de streaming impose une interface plus dense côté enseignant, et exclut par construction la correction conversationnelle en cours de génération.

V2/Op. 2 — orchestrateur SDK *one-shot*. La variante `sdk-orchestrator` pousse le déplacement plus loin : l’unique surface est un endpoint REST qui répond HTTP 202 et exécute la génération en arrière-plan, en parallélisant les unités via la primitive `Send` de `LangGraph`. La planification produit un contrat de cours complet *avant* la première écriture, ce qui réduit la dérive structurelle. Cette variante est mieux adaptée à un *déclenchement industriel* (autre application, batch nocturne, ingestion programmatique). En contrepartie, l’enseignant n’a aucun levier pendant l’exécution : la seule observation est la machine d’état publique, comme en V2/Op. 1, et toute correction suppose de repasser par la surface atelier.

2.4.4 Coûts observés et enveloppes d’usage

Quelques mesures issues de l’instance déployée fixent les ordres de grandeur, sans constituer un classement. La génération d’un syllabus complet via la V1/Op. 1 prend, selon la complexité de la requête et l’activation ou non d’un cycle critique, de l’ordre de la demi-minute à deux minutes. La production d’une activité par l’agent 2 paie une phase de lecture qui ajoute de l’ordre de plusieurs dizaines de pour cent de latence à la ligne de base de l’agent 1, et environ triple la taille du prompt d’entrée. Les voies V2, qui n’exposent pas de latence interactive, se caractérisent plutôt par leur *débit agrégé* d’une vague, lequel dépend du parallélisme inter-unités et du plafond imposé par l’ordonnanceur.

Ces chiffres ne désignent pas une variante préférable : ils qualifient des *enveloppes d’usage* différentes. Une enveloppe interactive (V1) suppose une attente d’utilisateur de l’ordre de la minute et tolère le streaming. Une enveloppe atelier (V2/Op. 1) suppose une production asynchrone observable par statut. Une enveloppe industrielle (V2/Op. 2) suppose un déclencheur programmatique non bloquant. Chaque enveloppe est cohérente avec sa variante d’origine et incohérente avec les autres : c’est ce constat, plus que les valeurs absolues mesurées, qui motive le chapitre suivant.

2.4.5 Enseignements croisés

Trois enseignements émergent de cette lecture comparative. D'abord, la *propriété de l'orchestration* (LLM contre code) est l'axe structurant : il décide de l'auditabilité, de la reproductibilité et du coût d'intégration plus que le nombre de sous-agents ou la nature du critic. Ensuite, les variantes diffèrent moins par la qualité du contenu que par la *posture humaine* qu'elles supposent ; aucune n'est universellement meilleure parce qu'aucune ne s'adresse à la même phase du parcours pédagogique de SprintUp. Enfin, la valeur de l'indexation et du MCP n'est pas uniforme : elle est conditionnelle au périmètre de la requête, ce qui suggère qu'elle relève d'une stratégie de *routage* et non d'une activation inconditionnelle.

Ces trois constats — propriété de l'orchestration, posture humaine, conditionnalité de l'ancrage — ne se résolvent pas au niveau d'une variante isolée. Ils appellent une *stratégie d'orchestration* qui articule la coexistence des variantes selon la phase du workflow et le contexte d'intégration. Le chapitre suivant prend en charge cette stratégie : routage entre variantes, articulation des surfaces conversationnelle, atelier et SDK, politique de sécurité adossée au principe du moindre privilège, et procédure d'opérationnalisation sur l'instance déployée.

2.5 Conclusion : du catalogue à l'orchestration

Ce chapitre a présenté la conception logique de la couche d'agents de SprintUp et la lecture comparative des implémentations livrées. Trois éléments le structurent. **L'indexation vectorielle** ne remplace pas la lecture directe des activités sœurs : elle la prolonge, comme garde-fou intra-syllabus et comme instrument principal d'anti-redondance et de recommandation à l'échelle de toute la base. **Le protocole MCP** standardise l'accès aux données et matérialise concrètement le principe du moindre privilège : trois serveurs disjoints, une liste blanche d'outils par agent, aucun chemin direct de l'agent vers la base de données. **Le catalogue des agents** — deux générateurs d'activités, quatre variantes du concepteur de syllabus, un tuteur, un assistant élève et un assistant administrateur — couvre l'ensemble du cycle pédagogique sans imposer un unique paradigme d'interaction.

La lecture comparative de la §2.4 a mis en évidence le point central de ce chapitre : les quatre variantes de l'agent 3 ne s'inscrivent pas sur une échelle linéaire de qualité, mais répondent à des questions d'intégration distinctes (co-construction conversationnelle, décomposition spécialiste, atelier asynchrone, déclenchement industriel). Aucune n'est universellement préférable, parce qu'aucune ne s'adresse à la même phase du workflow ni au même contexte d'usage. Cette pluralité est un constat de conception, pas une

indécision : elle reflète la diversité réelle des cas d’usage que SprintUp doit servir.

Cette pluralité ouvre, en retour, une question d’*orchestration de niveau supérieur*. Comment router une demande pédagogique vers la variante adaptée à la phase du parcours et au contexte d’intégration ? Comment articuler les surfaces conversationnelle, atelier et SDK sans démultiplier le code de plomberie ? Comment opérationnaliser, observer et sécuriser la coexistence de ces variantes sur l’instance déployée ? Et comment, sur cette même couche, traduire le principe du moindre privilège du MCP en une politique de sécurité applicative au sens propre ?

Le chapitre 3 prend ces questions en charge. Il définit la stratégie d’orchestration des variantes (politique de routage, articulation des surfaces, mutualisation des sous-graphes communs), décrit le dispositif d’observabilité associé (traces LLM, machine d’état publique, métriques de débit et de latence), et adosse à ce dispositif l’analyse de sécurité — surface d’attaque, contrôle d’accès, gestion des secrets, traitement des données personnelles — ainsi que les procédures d’opérationnalisation nécessaires à la mise en production. Le chapitre 2 a comparé les formes d’agents ; le chapitre 3 organise leur coexistence.

Chapitre 3

Stratégie d'orchestration et observabilité de la couche d'agents

Le chapitre 2 s'est achevé sur un constat à trois faces. La *propriété de l'orchestration* — selon qu'un superviseur LLM ou du code déterministe décide du prochain pas — est l'axe qui gouverne le plus directement l'auditabilité et la reproductibilité d'une variante. Les variantes diffèrent ensuite moins par la qualité intrinsèque du contenu produit que par la *posture humaine* qu'elles supposent : enseignant disponible en temps réel, opérateur qui pilote une production asynchrone, ou client programmatique sans humain dans la boucle. Enfin, la valeur de l'indexation et du protocole d'accès aux données n'est pas uniforme mais *conditionnelle au périmètre* de la demande. Aucune des variantes comparées n'est universellement préférable, parce qu'aucune ne s'adresse à la même phase du parcours pédagogique de SprintUp.

Cette pluralité n'est pas une indécision de conception : elle reflète la diversité réelle des usages que la plateforme doit servir. Elle déplace toutefois la question d'un niveau. Tant que l'on raisonne variante par variante, on décrit des formes d'agents isolées ; dès lors que ces formes coexistent, il faut une logique de niveau supérieur pour décider laquelle invoquer, pour observer ce qu'elles font une fois lancées, et pour reconnaître ce qu'elles ne savent pas encore faire ensemble. Le présent chapitre prend en charge cette logique. Il ne propose pas *une* orchestration unique qui remplacerait les variantes ; il propose une *stratégie* qui organise leur coexistence, en décrit l'observation et en énonce honnêtement les limites.

Le chapitre s'organise en quatre temps. La première partie définit la stratégie d'orchestration des variantes : un rappel resserré des trois surfaces d'interaction héritées du chapitre 2, la politique de routage qui choisit la variante adaptée à une situation, les sous-graphes que plusieurs variantes mutualisent, et la composabilité aujourd'hui

limitée de ces surfaces. La deuxième partie décrit ce que signifie *observer* cette couche d'agents en fonctionnement, à partir de trois artefacts d'observation et d'un identifiant de fil servant de trame de corrélation. La troisième partie dresse un bilan lucide des limites actuelles et des besoins de fiabilisation, au niveau de l'architecture et non de l'exploitation. La dernière partie fait le bilan du chapitre et prépare la transition vers le chapitre 4, qui traitera spécifiquement de la sécurité applicative de cette couche d'agents.

3.1 Stratégie d'orchestration des variantes

La thèse de cette partie tient en une phrase : il n'y a pas une orchestration de la couche d'agents, mais une stratégie qui choisit la variante adaptée à la phase du travail et au contexte d'intégration. Cette stratégie se lit sur trois plans complémentaires. Le premier est celui des *surfaces* par lesquelles une demande entre dans le système. Le deuxième est celui du *routage*, c'est-à-dire des signaux qui déterminent quelle variante répond à une demande donnée. Le troisième est celui de la *mutualisation*, qui décrit ce que les variantes partagent en interne pour éviter la duplication. La partie se termine sur une discussion franche de la composabilité : ces surfaces fonctionnent aujourd'hui en parallèle, mais ne se composent pas encore en flux de bout en bout.

3.1.1 Trois surfaces d'interaction

Le chapitre 2 a décrit en détail les variantes de l'agent 3 et les a relues à travers leurs profils d'intégration (voir la lecture comparative de la section 2.4). Nous n'en reprenons ici que la structure d'ensemble, sous l'angle qui importe pour l'orchestration : chaque variante expose une *surface d'interaction* distincte, c'est-à-dire une manière propre pour un acteur — humain ou logiciel — de déclencher une génération et d'en suivre le déroulement. Trois surfaces se dégagent, et elles ne sont pas redondantes : chacune répond à une posture humaine que les autres ne servent pas aussi bien.

Surface conversationnelle. La première surface est celle du dialogue. L'enseignant s'adresse à un agent dans un fil de discussion ; un protocole de streaming lui livre les artefacts au fur et à mesure de leur production, et les questions de l'agent prennent la forme d'interruptions structurées. Deux variantes l'incarnent : la voie conversationnelle à supervision LLM avec *Human-in-the-Loop* (la variante **syllabus-generator**, décrite à la section 2.3.4) et la voie à sous-agents spécialistes (la variante **deepagent** de la section 2.3.4). Toutes deux supposent un enseignant présent, prêt à clarifier une commande mal spécifiée ou à corriger une dérive en cours de génération. C'est la surface du *co-construire à voix haute*.

Surface atelier. La deuxième surface renverse cette posture. Plutôt que de dialoguer, l'enseignant définit d'abord la structure — unités et activités — sous forme de brouillons, puis déclenche explicitement la génération par vagues. La variante d'atelier (le couple `unit-bulk-supervisor` et `unit-activity-generator`, décrit à la section 2.3.4) ne pratique ni streaming de tokens ni interruption en ligne : la progression d'une activité n'est exposée que par une machine d'état publique. C'est la surface de l'*atelier rédactionnel asynchrone*, où l'opérateur décide quand lancer une vague et observe ensuite l'avancement à travers des statuts typés.

Surface programmatique. La troisième surface retire l'humain de la boucle d'exécution. Un système client déclenche la génération d'un syllabus par un appel programmatique, sans interface conversationnelle ni atelier. La variante d'orchestration par kit de développement (la variante `sdk-orchestrator`, décrite à la section 2.3.4) répond par construction à ce contrat : un appel non bloquant lance la génération en arrière-plan, et la seule observation disponible reste, là encore, la machine d'état publique. C'est la surface du *déclenchement industriel*, qui sert une autre application, un traitement par lots ou une ingestion automatisée.

Il est éclairant de relire ces trois surfaces à travers les axes de comparaison établis au chapitre 2 (voir la section 2.4.1), car la stratégie d'orchestration en hérite directement. La *propriété de l'orchestration* les sépare nettement : sur la surface conversationnelle, un superviseur LLM décide à chaque tour du prochain pas, ce qui maximise la souplesse au prix de la reproductibilité ; sur les surfaces atelier et programmatique, l'enchaînement des activités est porté par du code déterministe, ce qui rend l'exécution reproductible mais ferme la porte à l'adaptation en ligne. La *granularité de l'artefact inspectable* varie de la même façon : un flux de jetons éphémère sur la surface conversationnelle, des statuts typés et persistants sur les deux autres. Quant à la *posture humaine* — enseignant présent, opérateur asynchrone, client sans humain — elle constitue la différence la plus visible, et c'est elle que le routage doit reconnaître en premier. Choisir une surface, c'est donc choisir d'un même geste un mode d'orchestration, un grain d'observation et une place pour l'humain ; ces trois choix sont solidaires et ne se recombinaient pas librement.

Ces trois surfaces ne sont pas trois implémentations concurrentes d'un même besoin : elles répondent à trois moments distincts du travail pédagogique. La surface conversationnelle sert l'exploration et la clarification ; la surface atelier sert la production maîtrisée d'un volume connu ; la surface programmatique sert l'intégration dans un système tiers. La stratégie d'orchestration ne consiste donc pas à préférer l'une à l'autre, mais à reconnaître laquelle convient à la situation présente — ce qui est précisément l'objet du routage.

3.1.2 Politique de routage

Router, ici, signifie choisir la variante qui répondra à une demande. Il importe de préciser d'emblée le statut de ce qui suit : nous décrivons un *cadre de décision*, et non un moteur de routage automatique. Aujourd'hui, le routage est très majoritairement *explicite* — c'est l'utilisateur ou le système intégrateur qui choisit la surface, donc la variante. Le routage assisté, où le système suggérerait la variante adaptée, relève des perspectives et n'est pas implémenté. Cette honnêteté de cadrage est essentielle : il serait trompeur de présenter comme un algorithme sophistiqué ce qui est, pour l'essentiel, un choix laissé à l'acteur.

Le choix de la variante se laisse néanmoins formaliser comme une fonction de trois familles de signaux :

$$\text{routage}(\text{intention}, \text{contexte}, \text{contrat}) \longrightarrow \text{variante}.$$

La première famille est l'**intention déclarée** : ce que l'acteur veut faire et, indissociablement, la surface qu'il choisit pour le faire. Un enseignant qui souhaite co-construire un syllabus en dialoguant ouvre un fil conversationnel ; un enseignant qui veut produire en bloc une série d'activités déjà structurées passe par l'atelier. L'intention et la surface sont, dans l'état actuel du système, presque confondues, et c'est ce qui rend le routage explicite : déclarer l'intention, c'est déjà choisir la variante.

La deuxième famille est le **contexte de session**, qui module un choix à l'intérieur d'une surface donnée. La présence ou l'absence d'un syllabus déjà lié à la session en est l'exemple le plus net. Lorsque la génération d'activités s'inscrit dans un syllabus existant, l'agent outillé décrit au chapitre 2 (l'agent 2, décrit à la section 2.3.3) est pertinent : il peut consulter les activités sœurs et ancrer sa production dans le vocabulaire et les objectifs de l'unité. En l'absence d'un tel rattachement, le contexte ne fournit rien à ancrer, et l'on retombe naturellement sur le comportement du générateur sans outillage (l'agent 1, décrit à la section 2.3.2). Ce basculement reprend directement le constat de la valeur conditionnelle de l'ancrage établi à la section 2.4.2 : le contexte de session détermine s'il y a lieu de payer le coût de l'ancrage ou non.

La troisième famille est le **contrat d'intégration**, qui distingue une sollicitation interactive d'un appel programmatique. Une interface utilisateur interactive ouvre la voie aux surfaces conversationnelle et atelier ; un appel programmatique impose, par construction, la surface programmatique. Le contrat n'est pas un simple détail d'implémentation : il conditionne la présence ou non d'un humain susceptible de répondre à une interruption, et donc la possibilité même d'un dialogue en cours de génération.

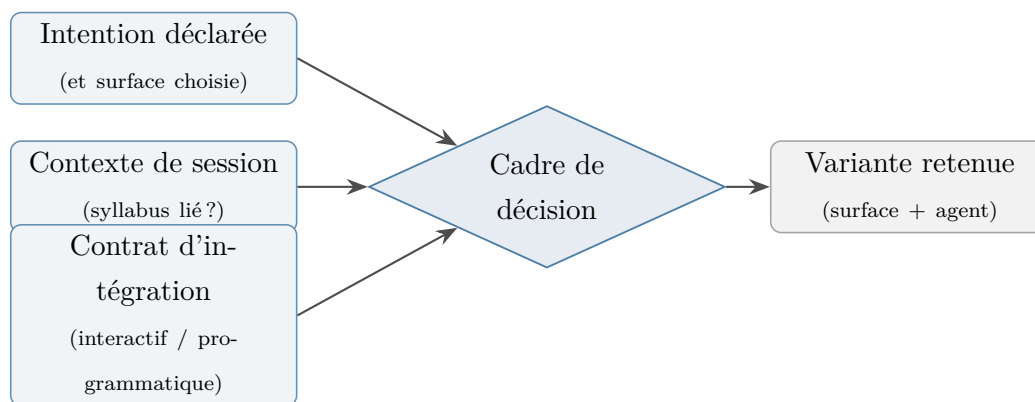


FIGURE 3.1 : Cadre de décision du routage : trois familles de signaux (intention déclarée, contexte de session, contrat d'intégration) déterminent la variante invoquée. Le cadre est aujourd'hui appliqué de manière explicite par l'acteur ; sa formalisation prépare un routage assisté laissé en perspective.

La figure 3.1 résume ce cadre. Il faut en lire la flèche centrale pour ce qu'elle est : une décision aujourd'hui prise par un acteur humain ou par un système intégrateur, non par un composant logiciel dédié. Formaliser le routage comme une fonction présente malgré tout deux intérêts. D'une part, cela rend explicites les signaux qui *devraient* gouverner le choix, ce qui aide à documenter et à enseigner l'usage de la couche d'agents. D'autre part, cela trace la voie d'un futur routage assisté : un composant pourrait, à terme, suggérer une variante à partir de l'intention exprimée et du contexte de session, sans retirer à l'enseignant la décision finale. Cette évolution est inscrite aux perspectives et n'est pas présentée ici comme un acquis.

Quelques situations concrètes illustrent l'application de ce cadre. Un enseignant qui dispose d'une commande encore floue — une discipline, un niveau, mais aucun plan arrêté — a intérêt à ouvrir un fil conversationnel, où l'agent l'aidera à clarifier l'intention avant de produire : l'intention déclarée pointe vers la surface conversationnelle. Un enseignant qui a déjà saisi une douzaine d'activités en brouillon et souhaite les voir générées d'un bloc relève au contraire de la surface atelier, où il déclenchera une vague et en suivra l'avancement par statuts. Une application tierce qui veut produire un grand nombre de syllabus sans intervention humaine relève de la surface programmatique, par contrat d'intégration. Et, à l'intérieur d'une génération d'activités, le contexte de session tranche entre l'agent outillé et l'agent sans outillage selon que la demande s'inscrit ou non dans un syllabus lié — c'est-à-dire selon qu'il y a, ou non, un existant à consulter et à ne pas dupliquer.

Ces signaux ne sont pas toujours indépendants, et leur articulation suit une forme de préséance. Le contrat d'intégration prime : un appel programmatique exclut par construction les surfaces interactives, quelle que soit l'intention exprimée, car aucun

humain n'est disponible pour répondre à une interruption. Une fois la surface fixée par l'intention et le contrat, le contexte de session intervient à un grain plus fin, pour sélectionner l'agent et le degré d'ancrage. Reconnaître cette préséance n'ajoute aucune automatisation : elle décrit simplement l'ordre dans lequel les signaux contraignent le choix, et ce que devrait respecter un futur routage assisté pour rester cohérent avec l'usage actuel.

3.1.3 Sous-graphes communs et mutualisation

Si aucune variante n'est universelle, alors le système doit faire coexister plusieurs formes d'agents. La contrepartie technique de ce constat est claire : pour éviter de réécrire quatre fois la même logique, les variantes doivent partager leurs briques internes. La mutualisation des sous-graphes est le pendant concret de la conclusion « aucune variante n'est universelle » : la diversité s'exprime au niveau des surfaces et des postures, pas au niveau de chaque opération élémentaire.

Trois briques se prêtent particulièrement à ce partage. La première est le **sous-graphe de génération d'activité**, qui enchaîne la planification d'une activité, sa rédaction, puis son indexation dans la base sémantique. Cette séquence constitue l'unité de travail élémentaire commune à la surface atelier et à la surface programmatique : dans les deux cas, le résultat n'est pas un flux conversationnel mais une activité persistée, dont l'avancement est exposé par la machine d'état publique. La voie conversationnelle à sous-agents mobilise la même logique à travers son sous-agent rédacteur. Extraire ce sous-graphe en une brique réutilisable évite que chaque surface réimplémente, à sa manière, la même chaîne planifier-rédiger-indexer, avec le risque de divergences silencieuses entre variantes.

La deuxième brique est le **critic pédagogique**, ce sous-agent dont le rôle est d'examiner une production avant de la considérer comme acceptable, au regard de la barre de qualité définie pour l'agent concerné. Mutualiser le critic, c'est garantir qu'une activité produite par la surface atelier et la même activité produite par la surface programmatique sont jugées selon les mêmes exigences. C'est aussi concentrer en un seul point l'évolution de ces exigences : relever la barre de qualité ou affiner un critère se fait une fois, et bénéficie à toutes les surfaces qui invoquent le critic.

La troisième brique est l'**accès aux outils de lecture et l'indexation**. La consultation des activités existantes, la recherche par similarité — notamment l'outil `find_related_activities` — et l'écriture indexée constituent un socle partagé, exposé par le protocole d'accès aux données et soumis au principe du moindre privilège rappelé à la section 2.2.4. Ce socle est ce qui rend l'ancrage possible quel que soit le point d'entrée : que la demande arrive par dialogue, par atelier ou par appel program-

matique, c'est le même catalogue d'outils, soumis aux mêmes restrictions, qui sert de pont vers l'existant. Mutualiser cet accès, c'est aussi mutualiser la discipline de sécurité qui l'encadre — un point sur lequel le chapitre 4 reviendra.

L'intérêt de cette mutualisation est double. Sur le plan de la maintenance, factoriser un sous-graphe de rédaction ou un critic partagé réduit la surface de code à faire évoluer : une amélioration de l'anti-redondance ou de la barre de qualité profite simultanément à plusieurs surfaces. Sur le plan de la cohérence, le partage garantit qu'une même demande, traitée par des surfaces différentes, s'appuie sur les mêmes garde-fous d'ancrage et de qualité. La diversité des variantes ne se paie donc pas en divergence de comportement sur les opérations de base : elle se concentre là où elle apporte de la valeur, c'est-à-dire dans la posture d'interaction et la propriété de l'orchestration.

Il faut toutefois mesurer la portée de cette mutualisation. Partager un sous-graphe n'est pas composer les surfaces entre elles : c'est réutiliser une brique *à l'intérieur* d'une variante donnée. La partie sur la composabilité examinera précisément ce que la mutualisation ne règle pas, à savoir le chaînage des surfaces en un flux unique. Auparavant, un cas particulier de mutualisation mérite d'être traité à part, car il touche directement au coût de fonctionnement : le partage du contexte de session.

3.1.4 Mutualisation du contexte et moment de l'ancrage

La lecture comparative du chapitre 2 a laissé une question ouverte, qu'il revient à ce chapitre de reprendre. L'ancrage d'une activité sur l'existant a un coût observable — de l'ordre de plusieurs dizaines de pour cent de latence supplémentaire et un volume de jetons d'entrée sensiblement accru — et ce coût est entièrement imputable à la phase de lecture qui précède la génération (voir la section 2.4.2). La question n'est pas de savoir s'il faut payer ce coût, puisque l'ancrage conditionne la non-redondance ; elle est de savoir *quand* le payer : à chaque génération, ou une fois pour un ensemble de générations relevant d'une même session.

Dans l'état actuel de la conception, la lecture d'ancrage est recalculée *à chaque génération*. Ce choix est simple et correct : chaque activité produite consulte l'existant au moment où elle est produite, ce qui garantit qu'elle s'appuie sur l'état le plus récent du syllabus. Il a néanmoins une conséquence visible lorsqu'une même session enchaîne plusieurs générations sur le même syllabus — cas fréquent sur la surface atelier : le contexte de lecture, pour l'essentiel identique d'une activité à l'autre, est reconstitué autant de fois qu'il y a de générations, et le tarif de l'ancrage est acquitté à répétition.

Le levier architectural correspondant consiste à *mutualiser le contexte de session* : lire une fois le contexte pertinent d'un syllabus — activités sœurs, plan d'ensemble, objectifs

— puis le réutiliser pour les générations successives d'une même session, plutôt que de le reconstruire à chaque fois. Ce levier prolonge directement la logique de routage exposée plus haut : on ne paie l'ancrage que lorsqu'il apporte de la valeur, et, idéalement, on ne le paie qu'une fois par session cohérente plutôt qu'une fois par activité. Il s'agit d'une mise en cache du contexte au niveau de la couche d'agents, et non d'un mécanisme de stockage particulier : la notion reste ici architecturale.

Un tel partage soulève en retour une question de fraîcheur. Un contexte mutualisé peut se périmer si des activités sont créées ou modifiées en cours de session ; sa réutilisation doit donc être bornée par une invalidation, naturellement adossée aux événements qui font évoluer le syllabus — transitions de la machine d'état des activités, nouvelles indexations. L'identifiant de fil introduit pour l'observabilité offre d'ailleurs une portée naturelle à ce contexte partagé : il délimite la session à l'intérieur de laquelle le contexte peut être réutilisé sans risque d'incohérence. Nous présentons ce levier comme une *perspective* cohérente avec la stratégie d'orchestration, et non comme un mécanisme déjà en place : la conception actuelle recalcule l'ancrage, et la mutualisation du contexte de session reste à concevoir.

3.1.5 Composabilité limitée

Une lecture honnête de la conception actuelle impose de reconnaître une limite : les surfaces sont *parallèles mais non composées*. Chacune offre un chemin complet, de la demande au résultat, mais aucune ne s'enchaîne automatiquement à une autre. Deux exemples concrets le montrent. Un déclenchement par la surface programmatique produit un syllabus complet, mais n'ouvre pas ensuite, de lui-même, un atelier de révision interactive : si l'enseignant souhaite reprendre le résultat à la main, il doit le rouvrir explicitement dans la surface atelier. Symétriquement, une session conversationnelle ne verse pas automatiquement son résultat dans un atelier de production par lots : le passage d'une surface à l'autre reste un geste manuel.

La cause de cette limite est structurelle plus qu'accidentelle : il n'existe pas, à ce jour, de graphe global unique qui porterait l'état d'un travail à travers les surfaces. Chaque surface gère son propre état — le fil de conversation pour les voies interactives, la machine d'état des activités pour les voies atelier et programmatique — sans qu'un orchestrateur de niveau supérieur ne relie ces états en une trajectoire continue. La conséquence est qu'un travail entamé sur une surface ne peut être prolongé sur une autre que par une intervention explicite, qui recommence depuis l'état observable de la surface d'arrivée.

Nous présentons ce point comme une limite assumée, et non comme une promesse non tenue. Composer les surfaces en flux de bout en bout — par exemple, déclencher

programmatically une génération puis basculer en atelier de révision sans perte d'état — supposerait un orchestrateur global et un état partagé que la conception actuelle ne fournit pas. Ce besoin est réel et il est inscrit aux perspectives ; il rejoint la discussion des limites de la partie consacrée à la fiabilisation, dans la troisième partie de ce chapitre. Le reconnaître clairement vaut mieux que de laisser croire à une composabilité qui n'existe pas encore.

On peut esquisser, sans entrer dans l'implémentation, ce qu'exigerait une telle composition. Il faudrait d'abord un *état partagé* qui survive au passage d'une surface à l'autre, de sorte qu'un travail entamé programmatically conserve son identité et son contenu lorsqu'il est repris en atelier. Il faudrait ensuite un *contrat de transfert* explicite entre surfaces, décrivant ce qu'une surface remet à la suivante et dans quel état le travail doit se trouver pour être repris sans ambiguïté. Il faudrait enfin un *orchestrateur de niveau supérieur* capable d'enchaîner ces transferts en respectant les postures humaines de chaque surface — par exemple, ne basculer en révision interactive que si un humain est effectivement disponible. Ces trois éléments dessinent un programme de travail cohérent ; la conception actuelle, qui privilégie des surfaces autonomes et bien délimitées, en constitue une fondation raisonnable plutôt qu'un obstacle.

3.1.6 Synthèse : une stratégie, non une orchestration unique

Les quatre plans qui précèdent dessinent ensemble une posture de conception cohérente, qu'il convient de résumer avant d'aborder l'observabilité. La stratégie d'orchestration ne cherche pas à fondre les variantes en un orchestrateur universel : elle les fait coexister en reconnaissant que chacune sert une posture d'usage distincte. Les *surfaces* encapsulent ces postures ; le *routing*, explicite aujourd'hui, choisit la surface et la variante à partir de l'intention, du contexte et du contrat ; la *mutualisation* garantit que cette diversité de surfaces ne se paie pas en duplication des opérations de base ni en divergence des garde-fous de qualité et d'ancrage ; la *composabilité*, enfin, demeure limitée et constitue le principal chantier ouvert.

Cette posture a une vertu méthodologique : elle distingue nettement ce qui relève d'un choix de conception assumé de ce qui relève d'une limite à lever. Faire coexister plusieurs surfaces est un choix, motivé par la pluralité réelle des usages de SprintUp. Ne pas les composer encore est une limite, dont la résolution suppose un état partagé et un orchestrateur de niveau supérieur. Tenir ces deux registres séparés — le choix et la limite — est précisément ce qui permet de défendre la conception sans la sur-vendre. C'est aussi ce qui prépare la partie suivante : une stratégie que l'on assume doit pouvoir être observée, de sorte que ses choix comme ses limites soient vérifiables en fonctionnement, et non seulement affirmés sur le papier.

3.2 Observabilité de la couche d'agents

Une stratégie d'orchestration n'est défendable que si l'on peut observer ce qu'elle produit en fonctionnement. Observer, ici, ne signifie pas déployer un outillage de supervision particulier — le choix d'un tel outillage sort du périmètre de la couche d'agents — mais identifier *ce qui* est observable, à quel niveau d'abstraction, et comment relier ces observations entre elles. Cette partie décrit trois artefacts d'observation indépendants de toute technologie, montre comment un identifiant de fil les relie, énumère les mesures opérationnelles utiles à suivre dans le temps, et propose un schéma de diagnostic par remontée. Conformément à la discipline adoptée dans tout le chapitre, nous distinguons à chaque étape ce qui est *déjà conçu et en place* de ce qui reste à *raffiner*.

3.2.1 Trois artefacts d'observation

Trois artefacts d'observation coexistent et se complètent. Pris séparément, chacun ne donne qu'une vue partielle ; c'est leur corrélation qui rend la couche d'agents intelligible.

Traces d'exécution. Le premier artefact est la trace d'exécution émise par les superviseurs et les sous-agents. Une telle trace est une séquence de phases et d'appels d'outils : sollicitation d'un outil de lecture, délégation à un sous-agent, passage du planificateur au rédacteur, appel du critic, et ainsi de suite. Ce niveau de détail est *en place* : la conception des variantes, en particulier les voies à supervision [LLM](#), produit naturellement ces traces, et elles sont la seule fenêtre sur le raisonnement interne des surfaces conversationnelles, qui n'exposent pas de machine d'état publique. La trace est l'artefact le plus riche, mais aussi le plus volumineux et le moins structuré : elle se lit a posteriori, pour comprendre *comment* un résultat a été obtenu.

La richesse de la trace est à la fois sa force et sa difficulté. Elle permet de reconstituer le cheminement exact d'une variante — quel outil a été appelé, dans quel ordre, avec quel effet — ce qui est indispensable pour les surfaces conversationnelles, où nul état public ne résume l'avancement. Mais cette richesse a un coût d'interprétation : une trace n'est pas un résumé, c'est un journal détaillé, et en tirer un diagnostic suppose de savoir quoi y chercher. C'est pourquoi la trace gagne à être lue *en regard* des deux autres artefacts plutôt qu'isolément, comme la suite de cette partie le montre.

Machine d'état publique des activités. Le deuxième artefact est la machine d'état publique des activités de la Version 2, déjà décrite et illustrée au chapitre 2 (voir la figure [2.5](#)). Une activité y progresse à travers une suite d'états — **draft**, **queued**, **searching**, **generating**, **ready** — assortie d'états terminaux d'échec — **failed** et **blocked**. Cet artefact présente une propriété décisive pour l'observabilité : il fait par-

tie du *contrat public* du système. Un client peut suivre l'avancement d'une génération en lisant uniquement ces états, sans interpréter le moindre protocole de streaming ni accéder au raisonnement interne de l'agent. La machine d'état est ainsi l'artefact le plus pauvre en détail mais le plus stable et le plus universellement consommable ; elle est *en place* pour les voies atelier et programmatique.

Cette pauvreté apparente est en réalité une vertu d'interface. Parce qu'elle ne dépend ni du modèle de langage employé, ni du détail de l'orchestration interne, la machine d'état offre une surface d'observation stable dans le temps : un client construit sur ces états ne se brise pas lorsque le raisonnement interne d'une variante évolue. Elle constitue ainsi le point d'observation de référence pour quiconque consomme la couche d'agents sans en connaître les rouages — et, à ce titre, le premier artefact que l'on consulte lors d'un diagnostic.

Mesures opérationnelles. Le troisième artefact est l'ensemble des mesures opérationnelles — latence, volume de jetons, fréquence d'erreurs — qui caractérisent une exécution indépendamment de la variante. Contrairement aux deux précédents, cet artefact relève aujourd'hui surtout du *préconisé* : l'architecture permet de produire et de collecter ces grandeurs, mais leur agrégation systématique dans un dispositif de suivi continu reste à construire. Nous le présentons donc comme un *dispositif d'observation à mettre en place*, et non comme un système de supervision déjà déployé. Les mesures elles-mêmes sont détaillées plus loin, dans la partie consacrée aux grandeurs utiles.

La complémentarité de ces trois artefacts tient à leurs niveaux d'abstraction. La machine d'état répond à la question « où en est ce travail ? » ; la trace répond à « par quel cheminement en est-il arrivé là ? » ; les mesures répondent à « à quel coût, et dans quelles marges de temps ? ». Aucun ne se substitue aux autres, et c'est leur mise en regard qui permet de raisonner sur l'architecture.

3.2.2 Corrélation par identifiant de fil

Pour relier ces trois artefacts, encore faut-il une clé commune. Cette clé est un *identifiant de fil* (ou identifiant de conversation), attribué à chaque travail entrant dans le système. Le même identifiant accompagne la demande de l'acteur, la trace d'exécution émise par les agents, et les transitions d'état des activités produites. Il joue le rôle de *fil conducteur* : à partir d'un seul identifiant, on peut remonter d'un symptôme observable jusqu'au détail du raisonnement qui l'a produit.

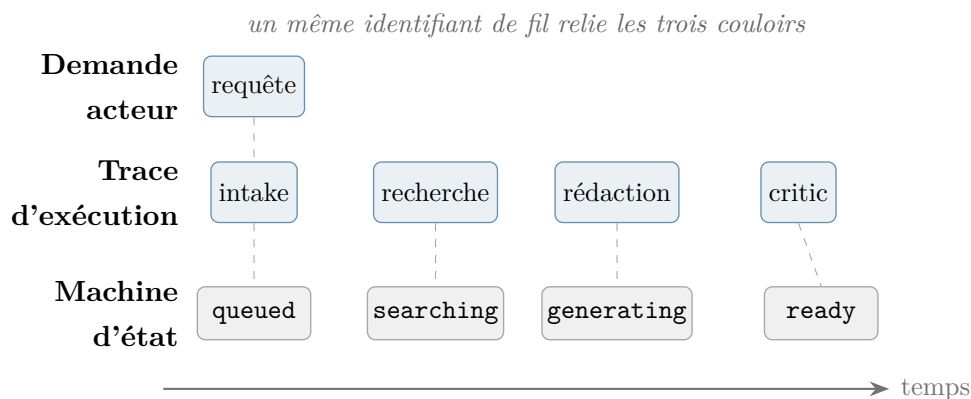


FIGURE 3.2 : Chronologie d'un fil. Une demande unique se déploie sur trois plans corrélés par un même identifiant de fil : la requête de l'acteur, la trace d'exécution (phases et appels d'outils) et la machine d'état publique des activités. La superposition permet de rattacher chaque état observable à la phase qui l'a produit.

La figure 3.2 illustre cette corrélation sous la forme d'une chronologie. Trois couloirs y avancent en parallèle le long d'un même axe de temps. Le premier porte la demande de l'acteur. Le deuxième déroule la trace d'exécution — intake, recherche, rédaction, critic — c'est-à-dire le raisonnement interne de la variante. Le troisième suit la machine d'état publique, dont les transitions (**queued**, **searching**, **generating**, **ready**) reflètent, vues de l'extérieur, l'avancement du travail. L'identifiant de fil, partagé par les trois couloirs, autorise la lecture verticale : à un état public donné correspond une phase de la trace, et à cette phase un coût et une durée mesurables. C'est cette lecture verticale qui transforme trois flux d'information séparés en un récit unique et exploitable d'un travail.

Il convient de situer le statut de ce dispositif. L'existence de traces détaillées au niveau des agents et celle d'une machine d'état publique sont *en place* : ce sont des propriétés de conception des variantes décrites au chapitre 2. La généralisation d'un identifiant de fil unique comme clé de corrélation systématique, à travers tous les artefacts et toutes les surfaces, relève en revanche du *préconisé* : le principe est posé, mais son application uniforme demande un travail d'instrumentation qui reste à mener.

3.2.3 Mesures opérationnelles utiles

Au-delà des artefacts qualitatifs, quelques grandeurs méritent d'être suivies dans le temps, car elles éclairent les compromis propres à chaque variante. L'argument n'est pas dans leur valeur absolue : il est dans leur *évolution* et dans ce qu'elle révèle des arbitrages de conception. Nous en retenons quatre.

- **Latence de bout en bout, par variante.** Le temps qui sépare le déclen-

chement d'un travail de la disponibilité de son résultat. Cette grandeur ne se compare pas d'une variante à l'autre dans l'absolu : une surface conversationnelle vise une attente de l'ordre de la minute et tolère le streaming, là où une surface programmatique répond de manière non bloquante et se qualifie plutôt par le débit d'une vague. La latence éclaire surtout l'adéquation d'une variante à l'enveloppe d'usage qu'elle sert, telle qu'elle a été décrite à la section 2.4.4.

- **Volume de jetons consommés par exécution.** La taille du contexte mobilisé pour produire un résultat. Le chapitre 2 a montré, à titre d'ordre de grandeur, que l'ancrage d'une activité sur l'existant accroît sensiblement le volume de jetons d'entrée par rapport à une génération non ancrée (voir la section 2.4.2). Suivre cette grandeur, c'est mesurer le *tarif de l'ancrage* et vérifier qu'il n'est payé que lorsqu'il apporte de la valeur.
- **Fréquence et nature des erreurs d'accès aux outils.** Les délais dépassés et les échecs d'appel aux outils de lecture ou d'écriture. Cette grandeur renseigne sur la robustesse de la couche d'accès aux données et, indirectement, sur la part des échecs de génération imputable non au raisonnement de l'agent mais à l'indisponibilité d'une ressource.
- **Temps passé par une activité dans chaque état.** La durée de séjour d'une activité dans les états de la machine publique — combien de temps en *queued*, en *searching*, en *generating*. Cette grandeur localise les goulets d'étranglement le long du flux nominal et distingue, par exemple, une attente d'ordonnancement d'une phase de recherche anormalement longue.

Ces grandeurs reprennent et prolongent les ordres de grandeur de la lecture comparative du chapitre 2. Il faut en rappeler le statut : l'objectif n'est pas de produire un classement chiffré des variantes, mais de *comprendre les compromis* — latence contre ancrage, débit contre contrôlabilité, richesse de la trace contre stabilité du contrat public. Suivre ces mesures dans le temps est précisément ce qui permettrait de faire de ces compromis un objet de décision, et non une simple intuition de conception.

Deux précautions encadrent l'usage de ces grandeurs. La première est de ne pas confondre une mesure ponctuelle avec une tendance : une latence observée sur quelques exécutions ne dit rien de fiable tant qu'elle n'est pas suivie dans la durée, et c'est précisément le suivi continu — aujourd'hui à mettre en place — qui donnerait son sens à ces chiffres. La seconde est de relier chaque grandeur à l'artefact qui l'explique : une latence anormale renvoie à la trace pour en localiser la phase coûteuse, un temps de séjour excessif dans un état renvoie à la machine d'état pour en circonscrire l'étape. Les mesures, en somme, ne se suffisent pas à elles-mêmes : elles signalent où regarder, et ce sont les deux autres artefacts qui disent quoi corriger.

3.2.4 Diagnostic par remontée

Les trois artefacts et leur clé de corrélation prennent tout leur sens dans une situation concrète : le diagnostic d'un symptôme. Le schéma proposé est une remontée à trois étages, de la surface observable vers la cause architecturale. Prenons un symptôme typique : une activité demeure longtemps dans l'état **queued**.

Le premier étage consiste à **inspecter l'état public**. La machine d'état indique sans ambiguïté que l'activité n'a pas quitté la file d'attente : le travail n'a pas échoué, il n'a pas non plus progressé. Cette observation, à elle seule, oriente déjà le diagnostic en écartant un échec franc (**failed**) ou un blocage propagé par dépendance (**blocked**).

Le deuxième étage consiste à **inspecter la trace du fil** correspondant. En suivant l'identifiant de fil, on relie l'activité bloquée à la séquence de phases qui la concerne, et l'on observe à quel moment la progression s'interrompt : la phase de recherche a-t-elle été lancée puis suspendue ? un appel d'outil est-il resté sans réponse ? l'ordonnancement a-t-il simplement différé le passage à l'état suivant ?

Le troisième étage consiste à **raisonner sur l'architecture**. Selon ce que révèle la trace, l'explication se rattache à un choix de conception identifiable : un sous-graphe de recherche particulièrement long sur un syllabus volumineux, un plafond de parallélisme imposé par l'ordonnanceur qui retient les activités en file, ou un délai d'accès à un outil de lecture. Le diagnostic ne s'arrête pas à constater le symptôme : il le rattache à une propriété de l'architecture, ce qui ouvre la voie à une correction raisonnée.

Le même patron s'applique à d'autres symptômes. Supposons une hausse de la fréquence des erreurs d'accès aux outils sur une période donnée. L'état public, là encore, fournit le premier indice : les activités concernées basculent en échec à la phase de recherche plutôt que de progresser. La trace du fil précise ensuite la nature de l'erreur — un délai dépassé, un appel resté sans réponse, plutôt qu'un refus de raisonnement de l'agent. Le raisonnement architectural conclut enfin en distinguant ce qui relève de la couche d'accès aux données de ce qui relève de la logique de l'agent : un échec concentré sur un même outil de lecture oriente vers la robustesse de cet accès, et non vers une révision du prompt de l'agent. La valeur du patron tient à cette capacité de *séparer les causes* — raisonnement, accès aux données, ordonnancement — au lieu de les confondre dans un symptôme global.

Ce schéma reste, à ce stade, un *patron conceptuel* : il décrit la manière de raisonner, non un tableau de bord prêt à l'emploi. La capacité d'inspecter l'état public et la trace d'un fil est en place ; l'outillage qui rendrait cette remontée immédiate et systématique — recherche par identifiant, mise en regard automatique des trois artefacts — reste à construire. C'est l'une des limites que la partie suivante examine.

3.2.5 Ce que l'observabilité permet de défendre

Il vaut la peine de récapituler ce que ce modèle d'observabilité autorise, en maintenant la distinction entre l'acquis et le projeté. Sont *en place*, parce qu'ils découlent directement de la conception décrite au chapitre 2 : l'existence de traces détaillées au niveau des superviseurs et des sous-agents ; la machine d'état publique des activités comme contrat d'observation stable ; et la possibilité, pour un fil donné, de mettre en regard un état public et la phase de trace correspondante. Ces propriétés suffisent à raisonner, au cas par cas, sur un comportement observé et à en remonter la cause.

Relèvent en revanche du *préconisé* : la généralisation d'un identifiant de fil unique comme clé de corrélation systématique ; la collecte continue et l'agrégation des mesures opérationnelles ; et l'outillage qui transformerait le diagnostic par remontée en un geste immédiat plutôt qu'en une investigation manuelle. La frontière entre ces deux registres n'est pas une faiblesse à dissimuler : c'est la description exacte de ce sur quoi une soutenance peut s'appuyer sans exagération. Affirmer que la couche d'agents *peut* être observée, et préciser à quel prix elle le serait de manière systématique, est plus solide que de revendiquer une supervision qui n'existe pas. Cette honnêteté de cadrage conduit naturellement à l'examen des limites, qui fait l'objet de la partie suivante.

3.3 Limites actuelles et besoins de fiabilisation

Les deux premières parties de ce chapitre ont décrit une stratégie d'orchestration et un dispositif d'observabilité. Il serait malhonnête de les présenter comme un système pleinement mûr. Cette partie dresse, au niveau de l'architecture et non de l'exploitation, un bilan lucide des limites actuelles et des besoins de fiabilisation. Trois familles de limites se dégagent : celles du routage et de la composition, celles de la gestion d'état et de la reprise, et celles de l'observabilité et des métriques.

3.3.1 Limites du routage et de la composition

La première limite a déjà été énoncée dans la partie sur l'orchestration, et il convient de la reprendre comme telle. Le routage demeure *explicite et manuel* : aucun composant ne combine automatiquement les signaux d'intention, de contexte et de contrat pour suggérer ou choisir une variante. Le cadre de décision formalisé à la section 3.1.2 décrit comment le choix *devrait* se faire, mais c'est aujourd'hui l'acteur — humain ou système intégrateur — qui l'exécute. Un routage assisté, qui proposerait une variante sans confisquer la décision finale, reste une perspective.

La seconde limite est celle de la composition. Les surfaces — conversationnelle, atelier, programmatique — ne se chaînent pas en flux à plusieurs étapes. Il n'existe pas

d'orchestrateur global qui enchaînerait, de manière sûre, un déclenchement programmatique suivi d'une révision en atelier, ou une exploration conversationnelle suivie d'une production par lots. Comme nous l'avons noté à la section 3.1.5, la cause est l'absence d'un graphe unique portant l'état d'un travail à travers les surfaces. Tant que cet état reste cloisonné par surface, la composition demeure un geste manuel plutôt qu'une trajectoire orchestrée.

Ces deux limites se rejoignent dans un même manque : celui d'un niveau d'orchestration qui se situerait *au-dessus* des variantes plutôt qu'à l'intérieur de chacune. Un routage assisté et une composition de surfaces sont en effet deux facettes d'un même besoin — décider, à partir de signaux, quelle variante invoquer, puis enchaîner plusieurs invocations en une trajectoire cohérente. Les énoncer ensemble n'est pas un aveu de faiblesse : c'est délimiter avec précision le périmètre de ce qui a été conçu (des surfaces autonomes et solides) et de ce qui reste à concevoir (leur coordination de niveau supérieur).

3.3.2 Limites de la gestion d'état et de la reprise

La fiabilité d'une couche d'agents se mesure aussi à sa capacité de reprendre un travail interrompu. Sur ce plan, la conception actuelle offre des possibilités réelles mais inégales selon la surface, qu'il faut décrire sans les surestimer.

Pour les voies conversationnelles, l'état d'un travail est porté par l'état du graphe de dialogue : une session interrompue peut, en principe, être reprise à partir de l'état conservé du fil, puisque le raisonnement progresse par tours successifs dont l'état est explicitement maintenu. Pour les voies atelier et programmatique, la reprise s'appuie sur la machine d'état des activités : une activité en échec peut être remise en file pour une nouvelle tentative, et la propagation d'un échec aux activités dépendantes est matérialisée par un état terminal dédié, comme l'illustre la figure 2.5. La granularité de la reprise est ici l'activité : on relance ce qui a échoué, sans nécessairement rejouer ce qui avait réussi.

Ces mécanismes ne constituent pas pour autant une reprise automatique et complète. En particulier, il n'existe pas de mécanisme de *retour arrière* automatique pour une chaîne d'actions complexe : annuler proprement un ensemble d'effets partiellement appliqués, lorsqu'un travail composite échoue à mi-parcours, n'est pas pris en charge de bout en bout et resterait à concevoir. La reprise actuelle est, pour l'essentiel, une reprise *en avant* — relancer ce qui a échoué — et non une annulation transactionnelle de ce qui a déjà été produit. Ce constat n'est pas un défaut surprenant pour un travail de cette nature ; il délimite simplement ce qui relève de l'ingénierie ultérieure.

3.3.3 Limites de l'observabilité et des métriques

La dernière famille de limites concerne l'observabilité elle-même. Il faut le reconnaître nettement : l'observabilité actuelle n'est pas *industrialisée*. Il n'existe pas de système complet et automatisé de supervision, avec tableaux de bord agrégés et alertes, qui suivrait en continu les grandeurs énumérées à la section 3.2.3. L'architecture rend ce suivi *possible* — les artefacts existent, la clé de corrélation est identifiée — mais le travail qui le rendrait systématique reste à faire.

Le mot « industrialiser » mérite d'être précisé, pour ne pas déborder sur un terrain qui n'est pas celui de ce travail. Il ne s'agit pas ici de choisir un outillage de supervision ni d'en décrire l'exploitation : ces questions relèvent de l'ingénierie de mise en œuvre et non de la conception de la couche d'agents. Il s'agit, plus modestement, de constater que les artefacts d'observation existent à l'état de *capacité* — on peut, au cas par cas, lire un état, suivre une trace, calculer une mesure — sans qu'une collecte régulière et une mise en regard automatique n'en fassent encore un instrument de pilotage. Combler cet écart est un travail d'instrumentation identifié, pas une lacune de conception.

Cette limite n'est pas isolée : elle prolonge celle du diagnostic par remontée, dont nous avons dit qu'il demeure un patron conceptuel faute d'outillage dédié. Elle prépare aussi une articulation naturelle avec le chapitre 4. La journalisation, l'auditabilité et la traçabilité ne sont pas seulement des instruments d'exploitation : ce sont aussi des exigences de sécurité. Savoir qui a déclenché quelle variante, sur quel périmètre, et pouvoir le retracer de manière fiable, relève autant de l'observabilité que du contrôle d'accès. Industrialiser l'observabilité et instruire la sécurité applicative sont ainsi deux chantiers qui se renforcent mutuellement, et le second fera l'objet du chapitre suivant.

En somme, ce que ce chapitre décrit est une *base architecturale solide* : des surfaces clairement caractérisées, un cadre de décision explicite, des artefacts d'observation complémentaires et une clé de corrélation identifiée. Ce n'est pas un système entièrement mûri et durci pour une exploitation à grande échelle — ce qui est normal, et même attendu, pour un projet de fin d'études. Nommer cette distinction n'affaiblit pas la conception : c'est au contraire la condition d'une défense honnête de ses apports comme de ses limites.

3.4 Bilan du chapitre et transition vers la sécurité applicative

Ce chapitre a prolongé la lecture comparative du chapitre 2 sur trois plans. Là où le chapitre 2 comparait des formes d'agents prises une à une, ce chapitre en a organisé la

coexistence.

Le premier apport est une **vue stratégique** de la manière dont les variantes coexistent et sont choisies. En distinguant trois surfaces d'interaction, en formalisant le routage comme une fonction de l'intention, du contexte et du contrat, et en identifiant les sous-graphes mutualisés, le chapitre a montré que la diversité des variantes n'est pas un éparpillement mais une réponse organisée à des postures d'usage distinctes. Il a aussi reconnu, sans détour, la composabilité encore limitée de ces surfaces.

Le deuxième apport est un **modèle conceptuel d'observabilité**, fondé sur trois artefacts — traces d'exécution, machine d'état publique, mesures opérationnelles — reliés par un identifiant de fil. Ce modèle fournit un langage pour raisonner sur le comportement de la couche d'agents en fonctionnement, depuis le suivi d'un travail jusqu'au diagnostic d'un symptôme par remontée.

Le troisième apport est une **vue honnête des limites** et des besoins de fiabilisation : routage explicite et non assisté, surfaces parallèles mais non composées, reprise en avant plutôt qu'annulation transactionnelle, observabilité possible mais non encore industrialisée. Ces limites tracent, en creux, le programme de travail d'une version ultérieure.

Au-delà de ces trois apports, le chapitre a maintenu une discipline transversale qui constitue en elle-même une contribution méthodologique : pour chaque surface, chaque artefact d'observation et chaque mécanisme de reprise, il a distingué ce qui est *en place* de ce qui est *préconisé*. Cette séparation n'est pas un simple souci de prudence rédactionnelle. Elle reflète la posture attendue d'un travail d'ingénierie : ne revendiquer comme acquis que ce qui est effectivement conçu et défendable, et formuler le reste comme une perspective argumentée. Appliquée à une couche d'agents — où il est facile de confondre ce que la technologie *permettrait* avec ce qui est *réalisé* — cette discipline est aussi une garantie d'honnêteté vis-à-vis du lecteur comme du jury.

Jusqu'ici, le rapport a suivi une progression continue. Les chapitres 1 et 2 ont posé la conception logique de la couche d'agents : fondations théoriques de l'orchestration et du protocole d'accès aux données, puis catalogue comparé des formes d'agents. Le présent chapitre y a ajouté la stratégie d'orchestration et le dispositif d'observabilité. Un axe demeure cependant à approfondir, que ce chapitre n'a abordé qu'en filigrane : la **sécurité applicative** de cette couche d'agents.

Cet axe mérite un traitement à part entière. Le principe du moindre privilège rappelé au chapitre 2 (voir la section [2.2.4](#)) constitue un point de départ, mais il ne dit rien de *qui* sollicite un agent ni de la manière dont ce contrôle se combine à une gestion des accès par rôle. De même, une couche d'agents fondée sur de grands modèles de

langage présente des surfaces d'attaque spécifiques — détournement par instructions injectées, fuite de données par les réponses — dont la mitigation n'a été ici qu'effleurée comme préoccupation, sans analyse dédiée. Le chapitre 4 prendra en charge cet axe. Il examinera comment le moindre privilège du protocole d'accès s'articule à un contrôle d'accès fondé sur les rôles, comment les injections d'instructions et les fuites de données se mitigent dans ce type d'architecture, et comment raisonner sur les contraintes de conformité dans ce contexte. Le chapitre suivant approfondit cet axe en examinant, sous l'angle de la cybersécurité, les surfaces d'attaque propres à une telle couche d'agents et les mesures de mitigation envisageables dans le cadre de SprintUp.

Conclusion générale et perspectives

Bilan du travail

Ce projet de fin d'études avait pour objectif de remplacer l'approche mono-modèle de génération de curricula par une **famille d'agents spécialisés** reliés à une base de connaissances pédagogique partagée à travers le Model Context Protocol. Le présent rapport se concentre sur la **conception logique** de cette famille d'agents et du serveur qui les relie, indépendamment de toute pile technique particulière. La coordination des agents par un orchestrateur ainsi que l'analyse de sécurité associée constituent des prolongements naturels de cette conception, qui seront traités dans des versions ultérieures du présent document.

Le travail s'organise autour de trois apports complémentaires. Le premier est un **mécanisme d'indexation** des activités pédagogiques, qui permet à un agent de consulter ce qui existe déjà avant de produire du contenu nouveau. Le deuxième est un **serveur MCP** qui expose ces données et les opérations associées sous la forme d'un catalogue d'outils typés, indépendamment de la nature des clients qui les consomment. Le troisième est un **catalogue d'agents** décrivant les rôles retenus pour cette première version, aux responsabilités distinctes et dont les privilèges sont restreints au strict nécessaire par une liste blanche d'outils [MCP](#).

Apports du projet

Les contributions principales de ce travail sont les suivantes.

Une architecture orientée qualité. Le déplacement du travail d'un seul modèle vers une famille d'agents spécialisés a été guidé par un objectif explicite, à savoir améliorer la qualité du contenu produit, et non en accélérer la production. La conception privilégie systématiquement l'ancrage des sorties sur des données existantes, la séparation des privilèges et la révision avant publication.

Une indexation pensée pour la non-redondance. Le mécanisme d'indexation par

plongement vectoriel n'est pas un ajout périphérique mais un composant de premier plan : il conditionne le comportement de l'agent générateur d'activités avec [MCP](#), qui consulte la base avant toute production. La représentation sémantique des activités et la recherche par similarité s'enchaînent pour faire de la non-redondance une propriété mesurable du système.

Un protocole MCP en pratique. L'usage du [MCP](#) permet de faire coexister, autour d'un même serveur, des agents aux profils très différents, sans dupliquer la logique d'accès aux données ni partager d'invite système. La discipline du contrat protocolaire facilite l'évolution future : ajouter un agent supplémentaire, par exemple l'assistant administrateur mentionné en perspective, ne demandera aucune modification du serveur ni des agents existants.

Une méthodologie d'évaluation centrée sur la qualité. La méthodologie de tests décrite au chapitre 2, partie 4, substitue aux mesures classiques de latence et de débit des mesures plus exigeantes : couverture des objectifs pédagogiques, progression interne des activités, justesse factuelle et non-redondance par rapport à la base. Elle accepte explicitement qu'un pipeline plus exigeant en qualité soit également plus exigeant en temps de calcul.

Limites

Plusieurs limites du présent travail méritent d'être signalées.

D'abord, l'**assistant administrateur**, mentionné en perspective, n'est pas inclus dans la présente version. Sa conception reprendra les principes des agents en lecture seule (tuteur et assistant élève) mais étendus à des opérations de supervision sur l'ensemble du curriculum.

Ensuite, le **choix définitif du modèle d'embedding** reste ouvert au moment de la rédaction. Les compromis entre modèle externe à [API](#) et modèle local exécuté en interne ont été documentés au chapitre 2, mais l'arbitrage final dépendra de mesures de coût et de qualité à effectuer sur l'infrastructure définitive.

Enfin, les **chiffres de qualité** produits par la méthodologie de tests ne sont pas reportés dans la présente version du document. Ils seront produits lors d'une phase d'évaluation dédiée et viendront compléter une version ultérieure du rapport.

Perspectives

Plusieurs directions s'ouvrent pour la suite du travail.

La première concerne la **conception de l'orchestrateur**, qui prendra en charge la coordination des agents au moment de l'exécution. L'algorithme de routage, le compteur de tours et le critère d'arrêt qui décident, à chaque tour de conversation, quel agent doit prendre la parole feront l'objet d'un travail dédié, à mener une fois la conception logique du présent document validée.

La deuxième concerne l'**ajout d'un agent supplémentaire**, l'assistant administrateur, dont la fonction de supervision globale du curriculum complètera la famille existante. Sa conception suit celle des agents en lecture seule (tuteur et assistant élève) mais étendue à des opérations transversales : détection d'activités obsolètes, statistiques de couverture des objectifs, diagnostic d'incohérences entre unités.

La troisième concerne l'**évolution du modèle d'embedding**. Le modèle initial sera choisi pour son coût et sa simplicité, mais un modèle multilingue de plus haute dimension pourrait être justifié si le volume d'activités croît significativement, ou si la qualité du regroupement sémantique devenait limitante en phase d'évaluation.

La quatrième concerne la **personnalisation des agents par domaine**. Les fiches d'agent du chapitre 2 sont génériques. Une spécialisation par discipline (mathématiques, sciences, langues) permettrait de raffiner les rubriques d'évaluation qualitative et les heuristiques d'ancrage propres à chaque discipline.

La cinquième et dernière concerne l'**analyse de sécurité** appliquée à l'ensemble. Au-delà des risques classiques d'une architecture web, les systèmes agentiques introduisent des vecteurs propres (injection par contenu indexé, empoisonnement de l'index, exfiltration par recherche) qui méritent une étude approfondie une fois l'architecture logique validée.

En conclusion, ce projet montre qu'une famille d'agents spécialisés, reliés par un contrat protocolaire standardisé et disposant d'un mécanisme d'indexation commun, constitue une réponse architecturale cohérente au problème de la génération pédagogique. La valeur de cette architecture se mesure d'abord à la qualité du contenu produit, à sa cohérence avec ce qui existe déjà, et à sa résistance aux entrées adversariales. Les chiffres qui supporteront cette affirmation seront produits par la méthodologie d'évaluation décrite au chapitre 2 et figureront dans une version ultérieure de ce document.

Références bibliographiques

- [1] I. GOODFELLOW et al., “Generative Adversarial Nets”, in *Advances in Neural Information Processing Systems*, t. 27, 2014.
- [2] A. VASWANI et al., “Attention Is All You Need”, in *Advances in Neural Information Processing Systems*, t. 30, 2017.
- [3] ANTHROPIC, *Introducing the Model Context Protocol*, <https://www.anthropic.com/news/model-context-protocol>, Consulté le 01/04/2026, nov. 2024.
- [4] ANTHROPIC, *Building Effective AI Agents*, <https://www.anthropic.com/research/building-effective-agents>, Consulté le 01/04/2026, déc. 2024.
- [5] LANGCHAIN, *LangGraph Documentation*, <https://langchain-ai.github.io/langgraph/>, Consulté le 01/04/2026, 2025.
- [6] LANGCHAIN, *LangGraph Supervisor Pattern*, https://langchain-ai.github.io/langgraph/tutorials/multi_agent/agent_supervisor/, Consulté le 01/04/2026, 2025.
- [7] IBM, *What is LangGraph ?*, <https://www.ibm.com/think/topics/langgraph>, Consulté le 01/04/2026, 2025.
- [8] ANTHROPIC, *What is the Model Context Protocol (MCP) ?*, <https://docs.anthropic.com/en/docs/agents-and-tools/mcp>, Consulté le 01/04/2026, 2025.
- [9] MODEL CONTEXT PROTOCOL WORKING GROUP, *Specification, version 2025-06-18*, <https://modelcontextprotocol.io/specification/2025-06-18>, Consulté le 01/04/2026, 2025.
- [10] GISKARD, *Model Context Protocol : Understanding MCP Security Risks and Prevention Methods*, <https://www.giskard.ai/knowledge/model-context-protocol-understanding-mcp-security-risks-and-prevention-methods>, Consulté le 01/04/2026, 2026.
- [11] ARIZONA STATE UNIVERSITY, *AI Innovation Challenge*, <https://ai.asu.edu/AI-Innovation-Challenge>, Consulté le 01/04/2026, 2024.

- [12] X. LI, Y. WANG et H. ZHANG, “Retrieval-Augmented Generation for Educational Application : A Systematic Survey”, *Computers and Education : Artificial Intelligence*, 2025. adresse : <https://scholars.ln.edu.hk/en/publications/retrieval-augmented-generation-for-educational-application-a-syst>
- [13] X. LI, Y. WANG et H. ZHANG, *RAG for Educational Application*, <https://www.scribd.com/document/897610174/4-RAG-for-Educational-Application>, Consulté le 01/04/2026, 2025.
- [14] INTELLECT EDUCATION, *AI-enhanced curriculum development at Intellect Academy using the SprintUp platform*, <https://sprintup.education/>, Consulté le 01/04/2026, 2025.
- [15] NORTHEASTERN UNIVERSITY, “Anthropic AI Partnership across campuses”, Northeastern University, rapp. tech., 2025.
- [16] DUKE UNIVERSITY, “Secure GPT-4 rollout with agentic capabilities”, Duke University, rapp. tech., 2025.
- [17] A. SINGH, M. EHTESHAM et al., “Agentic Retrieval-Augmented Generation : A Survey on Agentic RAG”, *arXiv preprint arXiv :2501.09136*, 2025. adresse : <https://arxiv.org/abs/2501.09136>
- [18] F. GAO et al., “KA-RAG : Integrating Knowledge Graphs and Agentic Retrieval-Augmented Generation”, *Applied Sciences*, 2025.
- [19] P. LEWIS, E. PEREZ, A. PIKTUS, F. PETRONI, V. KARPUKHIN, N. GOYAL et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”, in *Advances in Neural Information Processing Systems*, t. 33, 2020.